

GCI

Efficient Data Manipulation Using Numpy



- Review: Basics of Python
- Introduction to Numpy
- 1-Dimensional Arrays
 - Universal Functions, Broadcasting, and Aggregation Functions
 - Indexing
- 2-Dimensional Arrays
 - Axis
 - Basic/Advanced Index Specification

Topics Covered in this Lecture



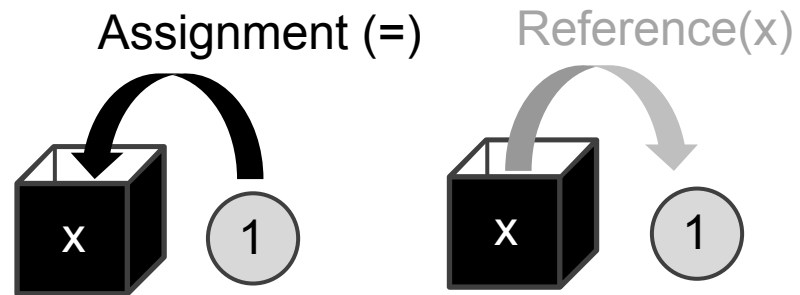
	Covered	Not covered
Array dimensions	1D・2D arrays	3D+ arrays
Level of complexity	Basic Numpy operations (refer to official Numpty doc)	Complex Numpy operations (defining machine learning models, etc.)
Notebook for practice	Basic operations and functions for exercises	Detailed explanation of linear algebra

Review: Basics of Python

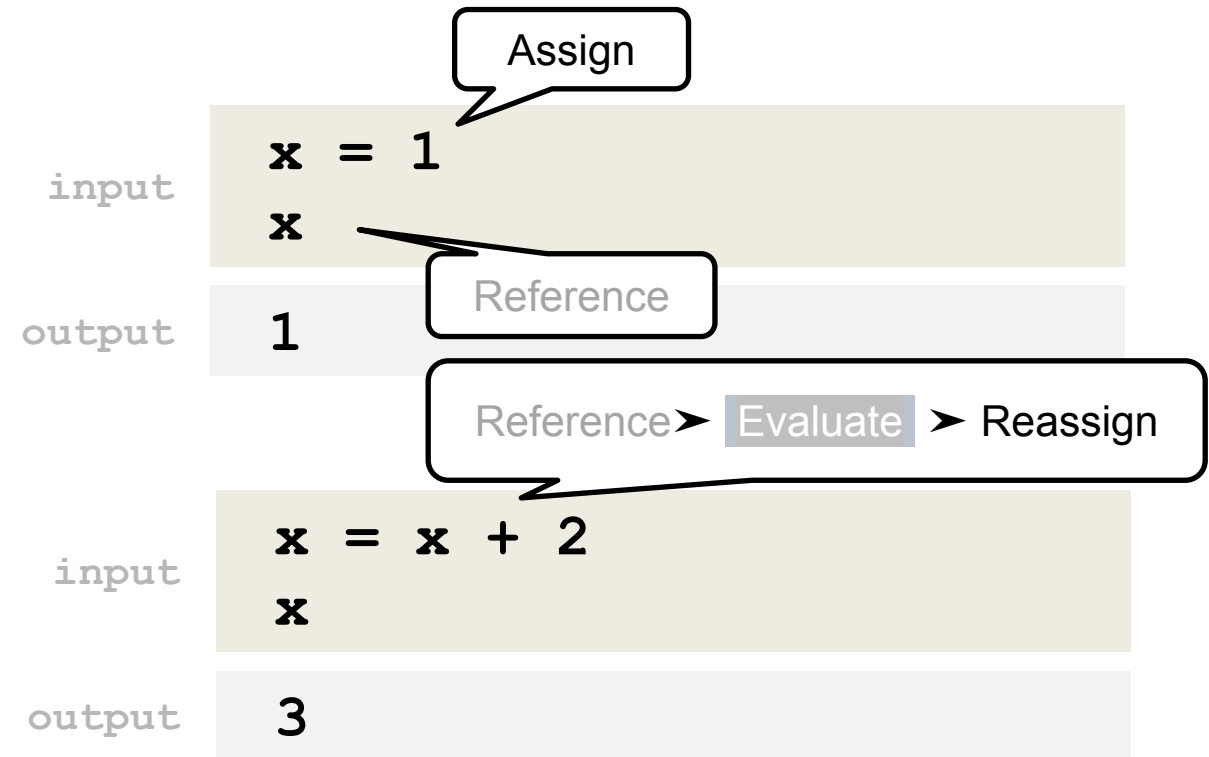
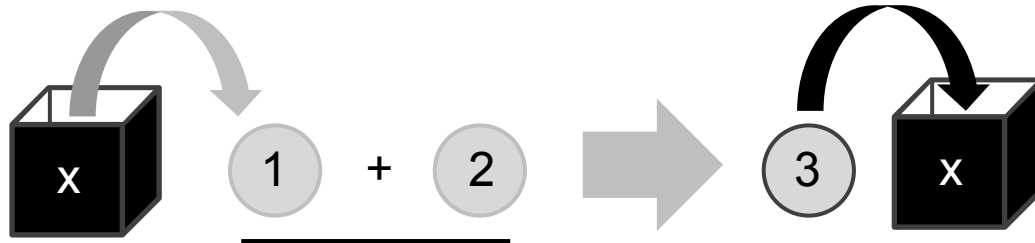
Variables



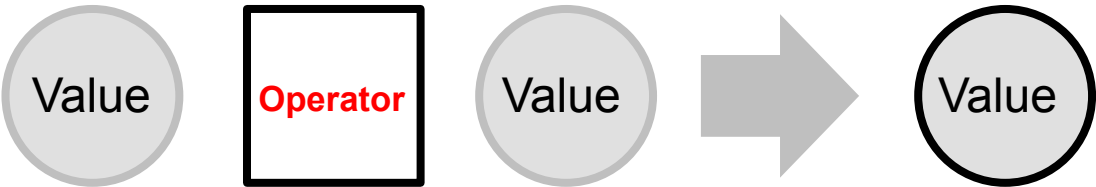
By setting **variable name = value**, the data on the right side is assigned to the variable on the left side.



Variable Calculation $x = x + 2$



An **operator** performs a calculation on values and returns the result.



Arithmetic Operators

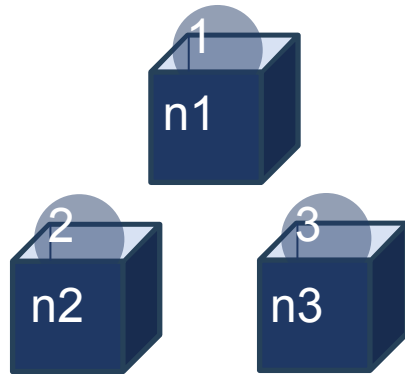
Symbol	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
//	Division (quetient)
%	Division (remaininder)
**	Powers

Comparison Operators

Symbol	Meaning
==	Left and right values are the same
!=	Left and right values are not the same
>	Left values are larger than right values
<	Left values are smaller than right values
>=	Left is greater than or equal to the right value
<=	Left is less than or equal to the right value

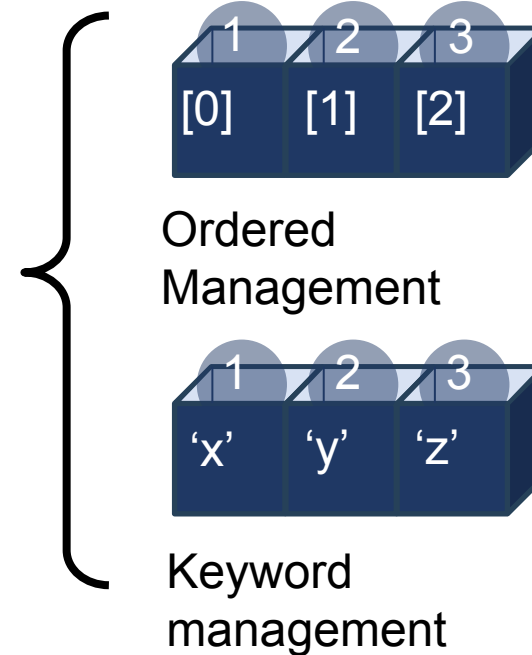
Collections allow you to manage values together.
There are types of **lists**, **tuples**, **dictionaries**, etc.

Assigning Values to Each Variable



Summary

Assigning Values to Collections



Example

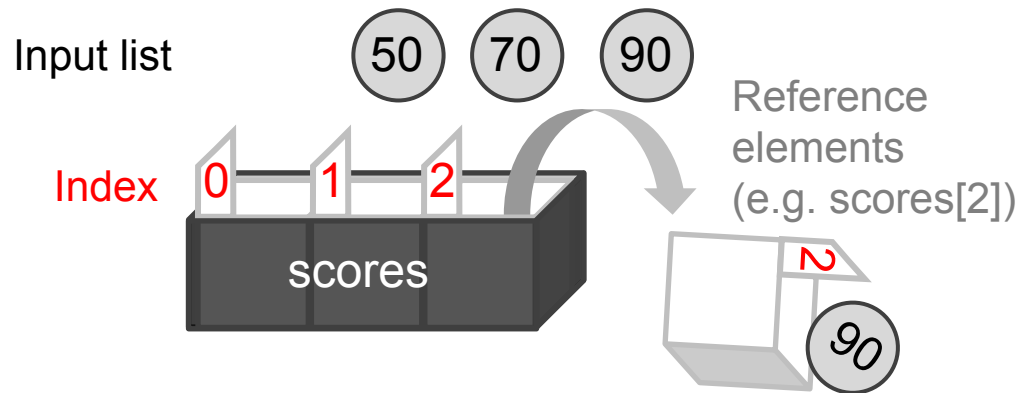
- Lists
- Tuples



- Dictionaries



A **list** is a data type that manages multiple values (**elements**) in sequence using **indexes**.



input

```
scores = [50, 70, 90]  
scores[2]
```

output

90

Assigns score "0" to
Index 2

input

```
scores[2] = 0  
scores
```

output

[50, 70, 0]

Calculates the sum

input

```
sum(scores)
```

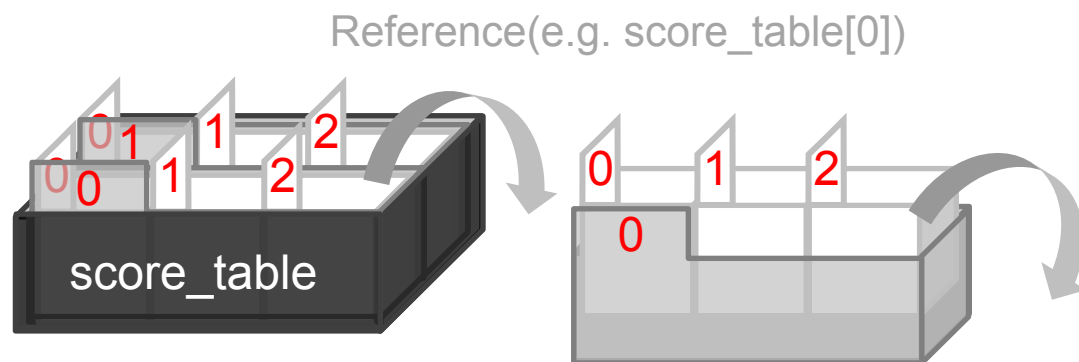
output

120

Two-Dimensional List



A list of lists. Manages values in a tabular form. Reference or modify elements by specifying the row and column indices.



input

```
score_table = [['A',  
'B', 'C'], [50, 70,  
90]]  
score_table[0]
```

names = ['A', 'B', 'C']
scores = [50, 70, 90]
score_table = [names,
scores]

output

```
['A', 'B', 'C']
```

In tabular form

	Student number	Column[0]	Column[1]	Column[2]
Name	Row [0]	['A',	'B',	'C'],
Score	Row [1]	[50,	70,	90]]

input

```
score_table[0][2]
```

output

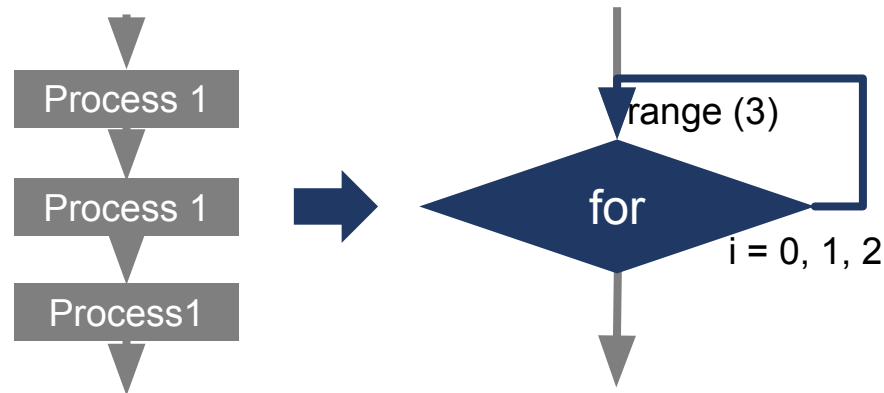
```
'C'
```

Row 0 Column
2

for Statement



Used when you want to repeat the same process without writing it over and over again. You can specify ranges and repeat them.



input

```
for i in range(0, 3):  
    print("Hello", i)
```

output

```
Hello 0  
Hello 1  
Hello 2
```

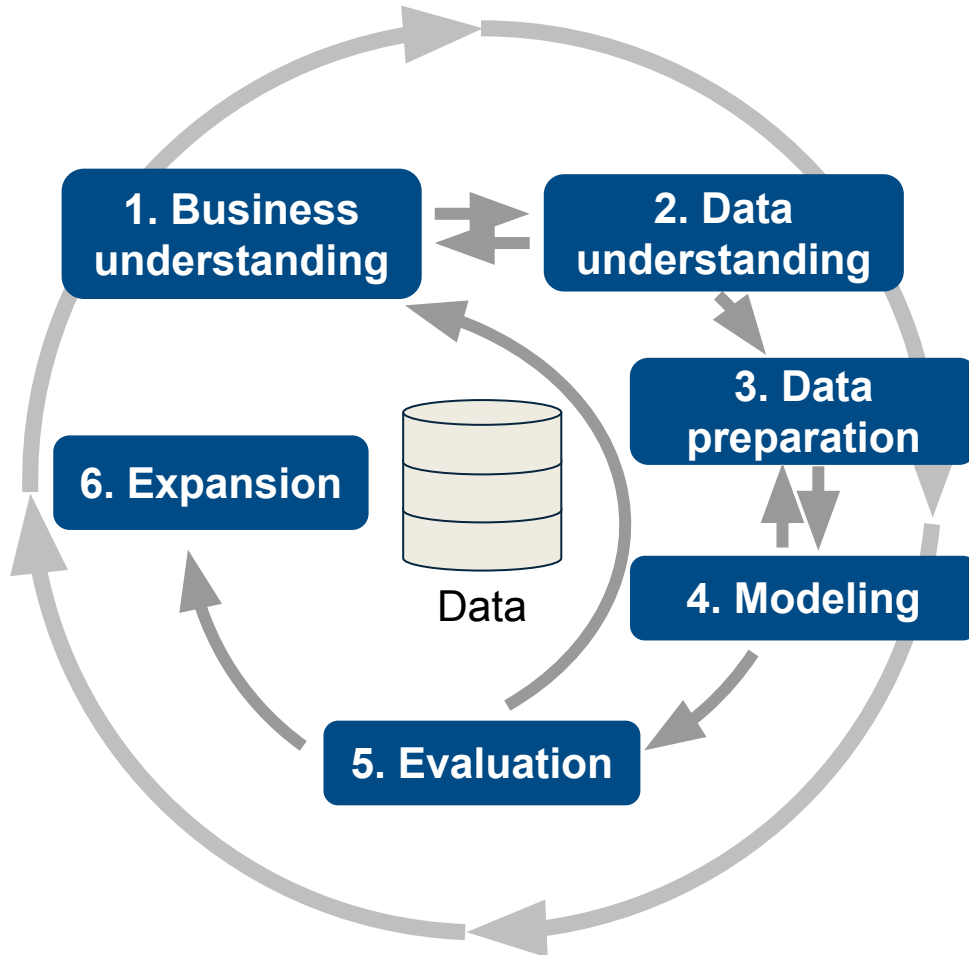
Introduction to Numpy

Data Analysis and Numpy



Data mining is the process of discovering and extracting useful knowledge and patterns from large datasets.

Goal of this class: **Learn basic data manipulation using Numpy**



Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C. and Wirth, R. (2000), CRISP-DM 1.0: Step-by-step data mining guide, CRISP-DM Consortium.

Data Analysis and Numpy



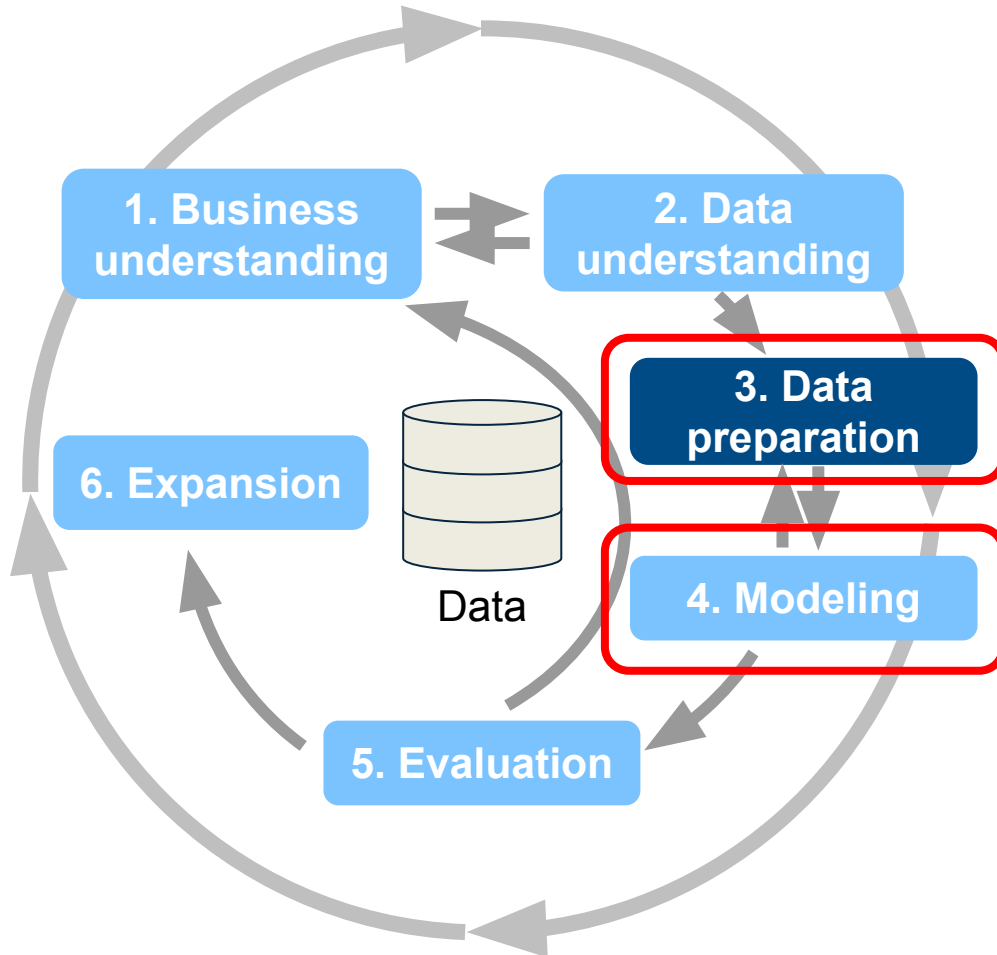
Data mining is the process of discovering and extracting useful knowledge and patterns from large datasets.

Goal of this class: **Learn basic data manipulation using Numpy**

3. Data preparation

Prepare & transform the data into a format suitable for analysis and modeling

Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C. and Wirth, R. (2000), CRISP-DM 1.0: Step-by-step data mining guide, CRISP-DM Consortium.



What is Numpy?



A library* to perform **complex scientific computations** in Python.

Characteristics of Numpy

1. Using `numpy.ndarray`, you can efficiently handle N-dimensional arrays
2. **Universal functions** allow complex calculations without using loops
3. Fast calculations (Numpy is mainly implemented in the high-performance C language)
4. A wide range of functions for scientific computing

*Library: A collection of tools and functions that are useful when writing programs

Numpy vs. Python list



A library* to perform **complex scientific computations** in Python.

2. **Universal functions** allow complex calculations without using loops.

When performing calculations with **Numpy**

input

```
sum = a + b
```

} Requires less codes!

When performing calculations with **lists**

input

```
sum = []  
for i in range(len(a)) :  
    sum.append(a[i] + b[i])
```

}

How to import Numpy?



Import Numpy using “import”

Please import as follows.

input

```
import numpy as np
```

*As a convention, Numpy is abbreviated as np.

Specify the abbreviation with `as` ○○

How to import Numpy?

Access numpy's functions using `np.○○({object to be processed})`

- Specify various function names in the `○○` part and apply the operation to the object inside the `()`.
- For example, it can be used as follows.

`input`

```
array_a = np.array([5, 6, 8])
```

Check out the documentation



- Functions are something to be looked up, not to be something memorized.
- Numpy (and not just Numpy) provides a vast number of functions.
- If there's something you'd like to try, let's search for it in the official documentation or other sources!

The screenshot shows the NumPy documentation website. At the top left is the NumPy logo. To its right are navigation links: 'User Guide', 'API reference', 'Development', 'Release notes', and 'Learn'. Below the logo is a search bar with the placeholder text 'Search the docs ...'. On the left side, there is a sidebar with the heading 'GETTING STARTED' and links for 'What is NumPy?', 'Installation', 'NumPy quickstart', and 'NumPy: the absolute basics for beginners' (which is highlighted in blue). Below this is the heading 'FUNDAMENTALS AND USAGE'. The main content area is titled 'How to import NumPy' and contains the text: 'To access NumPy and its functions import it in your Python code like this:'. Below this text is a code block containing the line `import numpy as np`. Further down, it explains that the name is shortened to `np` for readability and recommends always using `import numpy as np`. At the bottom of the screenshot, there is a blue link: 'NumPy: the absolute basics for beginners — NumPy v1.26 Manual'.

1-Dimensional Arrays

Recap: Python Lists

Python list is a data type with multiple values using [].

- A **list** is a format that is enclosed in [] and contains multiple values separated by commas.
- Example:

input

```
list_a = [0, 1, 2, 3, 4]  
list_a
```

output

```
[0, 1, 2, 3, 4]
```

Creating Numpy Arrays

Create Numpy arrays from lists by using `np.array()`.

- In Numpy, arrays are handled using the data structure **numpy.ndarray**.
- An ndarray can be created with **`np.array(list)`**.

input

```
array_a = np.array([0, 1, 2, 3, 4])  
array_a
```

output

```
array([0, 1, 2, 3, 4])
```

Various Numpy functions can be applied to ndarrays.

Python Lists vs Numpy Arrays



Comparison of Python list and numpy.ndarray

	List	numpy.ndarray
Element Type	Can store values of different types	Can only store values of the same type
Arithmetic Operations	Arithmetic operations between lists are cumbersome	Arithmetic operations between ndarrays are easy
Handling Multi-dimensional Arrays	Can be simulated using nested structures, but it's cumbersome.	Can efficiently handle multi-dimensional arrays using the concept of "axis".

Universal Function (ufunc) | Arithmetic Operations

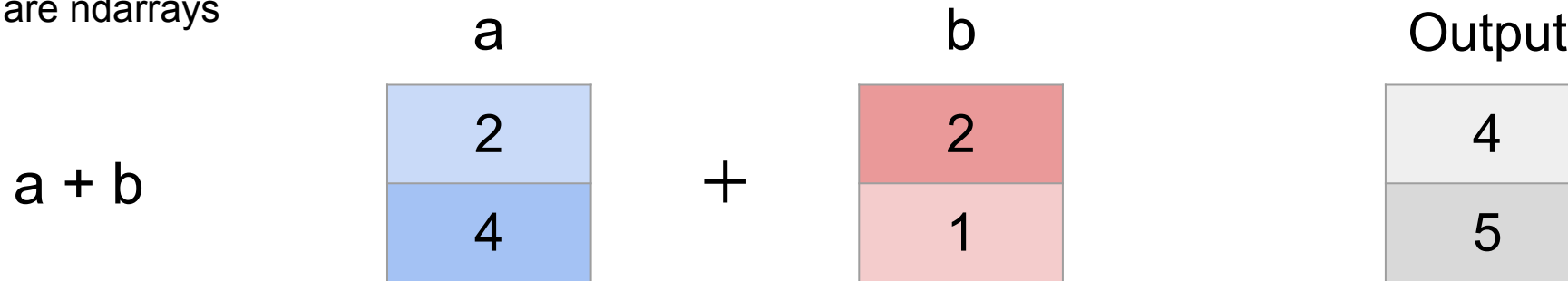


Universal Function: A function that operates on each element of an ndarray

- Most operations on ndarrays are applied between **elements at the same position**

- e.g. Addition

*a and b are ndarrays



Universal Function (ufunc) | Arithmetic Operations

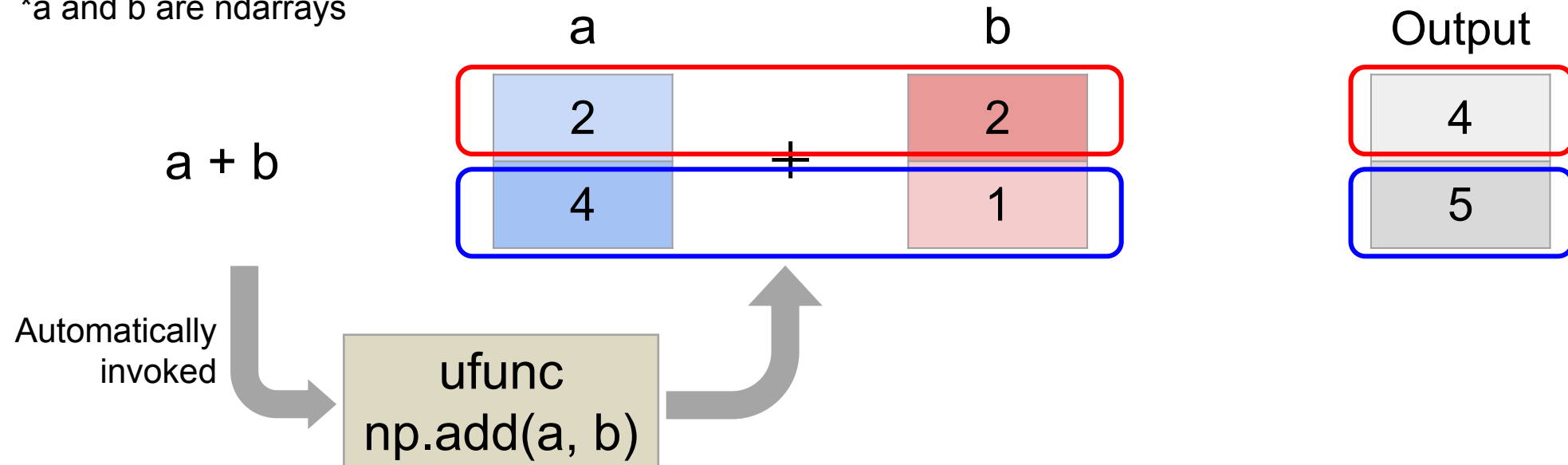


Universal Function: A function that operates on each element of an ndarray

- Most operations on ndarrays are applied between **elements at the same position**

- e.g. Addition

*a and b are ndarrays



Universal Function (ufunc) | Arithmetic Operations



Arithmetic operations on ndarray are all universal.

		a		b	Output
Subtraction np.subtract(a, b)	a - b	2	-	2	0
		4		1	3
Multiplication np.multiply(a, b) *This is not dot product or matrix multiplication	a * b	2	*	2	4
		4		1	4
Division np.divide(a, b)	a / b	2	/	2	1
		4		1	4

Universal Function (ufunc) | Arithmetic Operations



Arithmetic operations on ndarray are all universal.

- Code is supposed to look like the example below:
- e.g. Addition

input

```
a = np.array([2, 4])  
b = np.array([2, 1])  
a + b
```

output

```
array([4, 5])
```

Universal Function (ufunc) | More Mathematical Operations



Other mathematical functions on ndarray are all universal.

```
a = np.array([1, 2])
```

Then, the following output will be obtained.

Exponential
function

input

```
np.exp(a)
```

output

```
array([2.71828183, 7.3890561])
```

Logarithmic
function

input

```
np.log(a)
```

output

```
array([0., 0.69314718])
```

Trigonometric
functions

input

```
np.sin(a)
```

output

```
array([0.84147098, 0.90929743])
```

*The same applies to cos and tan

The elements of a are calculated as radians

Recap: “+” for Python Lists

“+” for Python lists are for list concatenation.

- Note: a for loop is required when performing arithmetic operations between elements of a list.
- When two lists are combined using the “+” operator, the lists are concatenated

input

```
a = [2, 4]
b = [2, 1]
a + b
```

output

```
[2, 4, 2, 1]
```

- Also, using arithmetic operators like “-”, “*”, “/” will result in an error.

Universal Function (ufunc) | Conditional Operations



Conditional operations on ndarray are all universal.

Equal Sign <code>np.equal(a, a)</code>	$a == a$	a 2 4	==	a 2 4	Output
					True
Equal Sign <code>np.equal(a, b)</code>	$a == b$	a 2 4	==	b 2 1	True
					False
Greater than <code>np.greater(a, b)</code> c.f. Less than: <code>np.less(a, b)</code>	$a > b$	a 2 4	>	b 2 1	False
					True

Recap: Conditional Operations on Python Lists



Conditional operations for Python lists compare the entire list.

- Note: When performing conditional operations on a list, the result returned applies to the entire list

input

```
a = [2, 4]
b = [2, 1]
a == a
```

output

True

input

```
a == b
```

output

False

Broadcasting: A feature that automatically adjusts shapes to enable operations

- The **scalar is extended to match the shape of the array**, and the operation is performed.

	a	Scalar		a	Array	Output
Addition	2	+	3	2	3	5
	4			4	3	7
Multiplication	2	*	1.6	2	1.6	3.2
	4			4	1.6	6.4

Aggregate Function



A function that aggregates the elements within a NumPy array into a single value

```
a = np.array([1, 2, 3])
```

Maximum value

input `np.max(a)`

output `3`

Sum of values

input `np.sum(a)`

output `6`

Average value

input `np.mean(a)`

output `2.0`

Standard deviation

input `np.std(a)`

output `0.816496580927726`

Indexing (1D Arrays) | Quick Quiz



Let's use indexing to extract values from wind speed data every 30 minutes.

Array a: Wind speed data for every 30 minutes.

12:00	12:30	13:00	13:30	14:00	14:30	15:00	15:30	16:00	16:30	17:00	17:30	18:00	18:30	19:00	19:30	20:00	20:30	21:00
12	13	15	14	9	11	10	13	9	7	8	-3	-5	-11	-9	-12	-8	0	1

Q1. How would you extract only the data for 13:00?

Q2. How would you extract the data from 13:00 to 16:00?

Q3. How would you extract data for every 60 minutes?



**Extract
using
indexing**

Indexing (1D Arrays)



Indexing: Specifying an index in an ndarray to extract specific elements.

- You can extract specific elements by specifying indices in the order 0, 1, 2, and so on from the left.

		[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]										
Array a	[	0	,	3	,	6	,	9	,	12	,	15	,	18	,	21	,	24	,	27	]

input

```
a = np.array([0, 3, 6, 9, 12, 15, 18, 21, 24, 27])  
a
```

output

```
array([0, 3, 6, 9, 12, 15, 18, 21, 24, 27])
```

Indexing (1D Arrays)



Indexing: Specifying an index in an ndarray to extract specific elements.

- You can extract specific elements by specifying indices in the order 0, 1, 2, and so on from the left.
- To extract only one value, specify only the start index.

a[1] ^[0] ^[1] ^[2] ^[3] ^[4] ^[5] ^[6] ^[7] ^[8] ^[9]
[0, 3, 6, 9, 12, 15, 18, 21, 24, 27]

input

a[1]

output

3

Indexing (1D Arrays)

Indexing: Specifying an index in an ndarray to extract specific elements.

- You can extract specific elements by specifying indices in the order 0, 1, 2, and so on from the left.
- To extract only one value, specify only the start index.
 - To retrieve the value at the last index, you can specify -1

a[-1]

	[-10]	[-9]	[-8]	[-7]	[-6]	[-5]	[-4]	[-3]	[-2]	[-1]
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
	[0,	3,	6,	9,	12,	15,	18,	21,	24,	27]

input

a[-1]

output

27

Indexing (1D Arrays)

Indexing: Specifying an index in an ndarray to extract specific elements.

- You can extract specific elements by specifying indices in the order 0, 1, 2, and so on from the left.
- To extract only one value, specify only the start index.
 - To retrieve the value at the last index, you can specify -1
- You can also specify multiple indices to retrieve values using lists.

a[[1, 4, 8]] ^[0] [0, ^[1] 3, ^[2] 6, ^[3] 9, ^[4] 12, ^[5] 15, ^[6] 18, ^[7] 21, ^[8] 24, ^[9] 27]

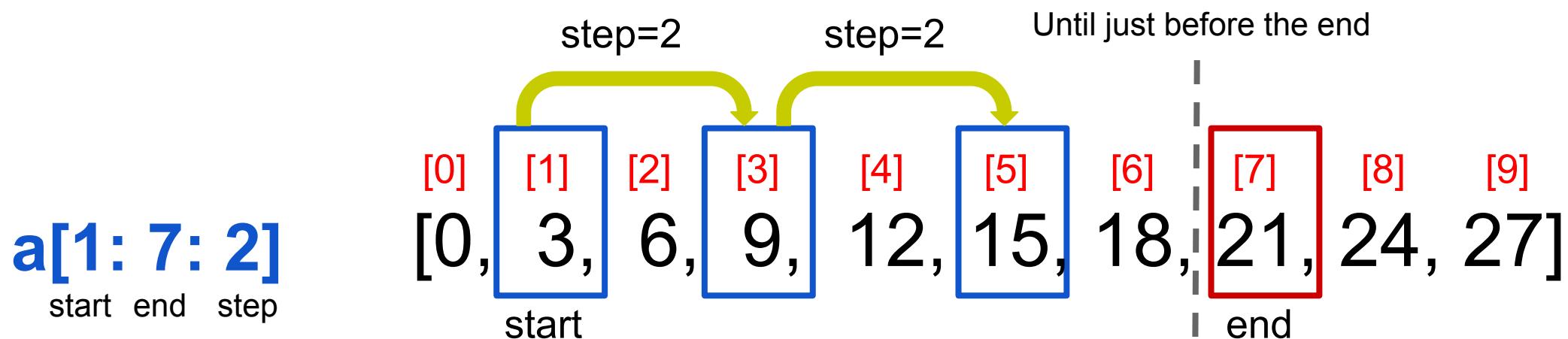
input `a[[1, 4, 8]]`

output `array([3, 12, 24])`

Slicing (1D Arrays)

Specify [start: end: step] in an ndarray to extract specific elements.

- To retrieve index values at regular intervals, specify **start**, **end**, and **step**
- Place a **slice** ":" between start, end, and step



input

```
a[1: 7: 2]
```

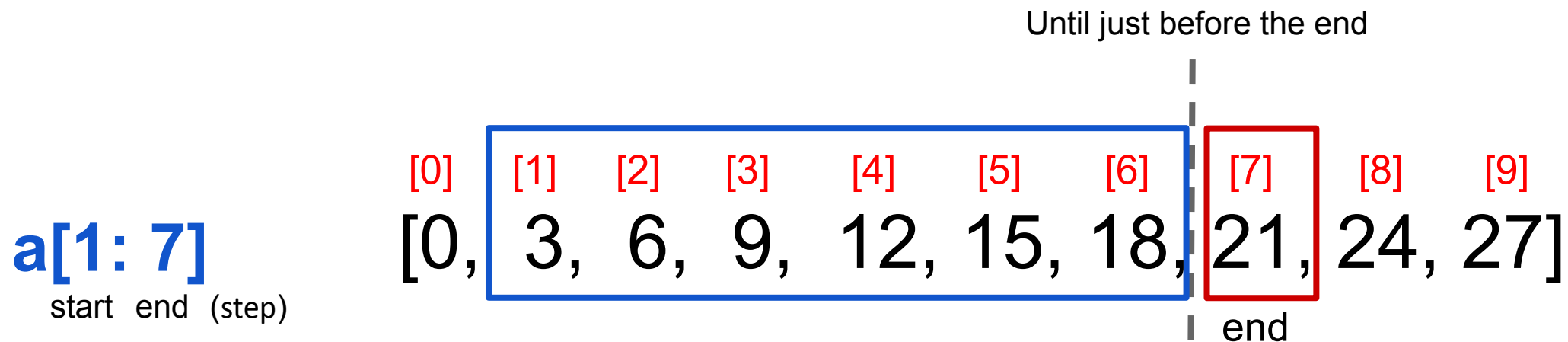
output

```
array([3, 9, 15])
```

Slicing (1D Arrays)

Specify [start: end: step] in an ndarray to extract specific elements.

- To retrieve a range of consecutive index values, specify the **start** and **end**.
- Place a **slice** ":" between start and end



input

`a[1: 7]`

output

`array([3, 6, 9, 12, 15, 18])`

Slicing (1D Arrays)

Specify [start: end: step] in an ndarray to extract specific elements.

- To retrieve all values after start, you can **omit end after the slice ":"**.

a[1:]
start (end)

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
[0,	3,	6,	9,	12,	15,	18,	21,	24,	27]

start

input

`a[1:]`

output

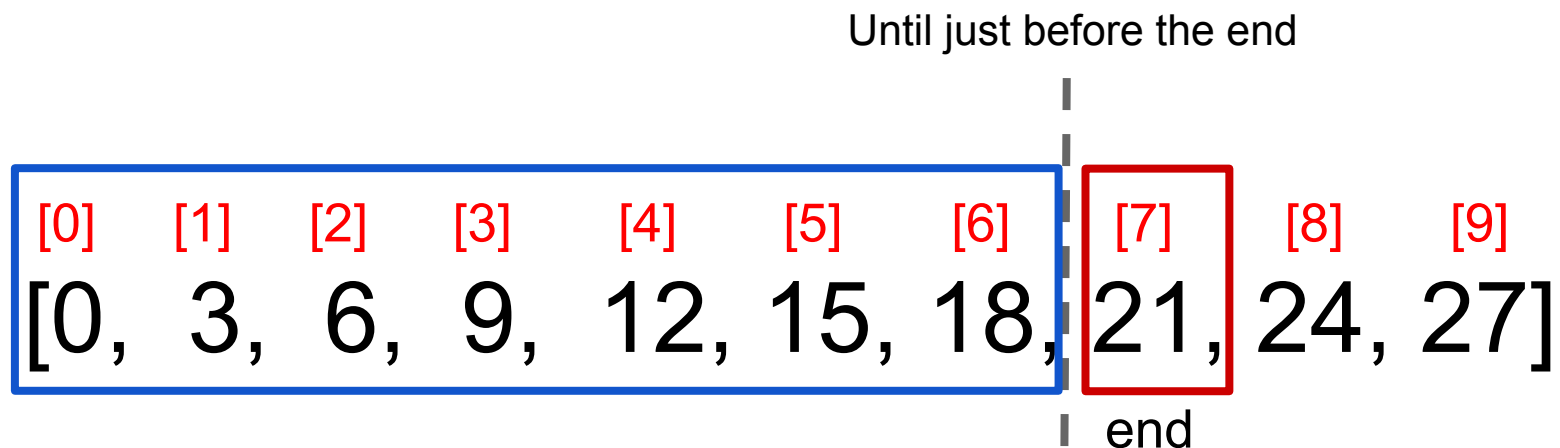
`array([3, 6, 9, 12, 15, 18, 21, 24, 27])`

Slicing (1D Arrays)

Specify [start: end: step] in an ndarray to extract specific elements.

- To retrieve all values before end, you can **omit start before the slice ":"**.

a[: 7]
(start) end



input

```
a[: 7]
```

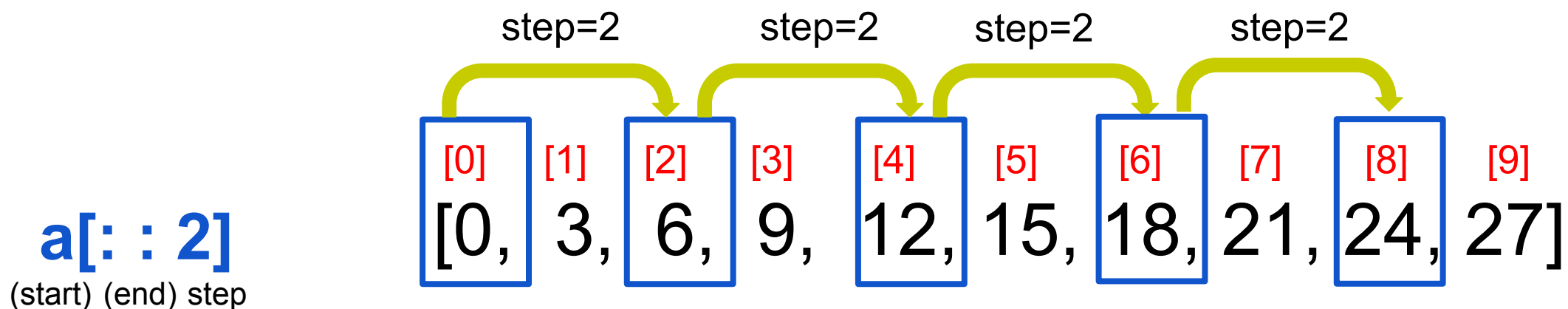
output

```
array([0, 3, 6, 9, 12, 15, 18])
```

Slicing (1D Arrays)

Specify [start: end: step] in an ndarray to extract specific elements.

- To retrieve index values at regular intervals across the entire array, specify **step**
- The start and end before and after the slice ":" can be omitted.



input

```
a[: : 2]
```

output

```
array([0, 6, 12, 18, 24])
```

Indexing (1D Arrays) | Quick Quiz Answer



Let's use indexing to extract values from wind speed data every 30 minutes.

Array a: Wind speed data for every 30 minutes.

12:00	12:30	13:00	13:30	14:00	14:30	15:00	15:30	16:00	16:30	17:00	17:30	18:00	18:30	19:00	19:30	20:00	20:30	21:00
[12,	13,	15,	14,	9,	11,	10,	13,	9,	7,	8,	-3,	-5,	-11,	-9,	-12,	-8,	0,	1]
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]	[13]	[14]	[15]	[16]	[17]	[18]

Q1. How would you extract only the data for 13:00?

input

`a[2]`

output

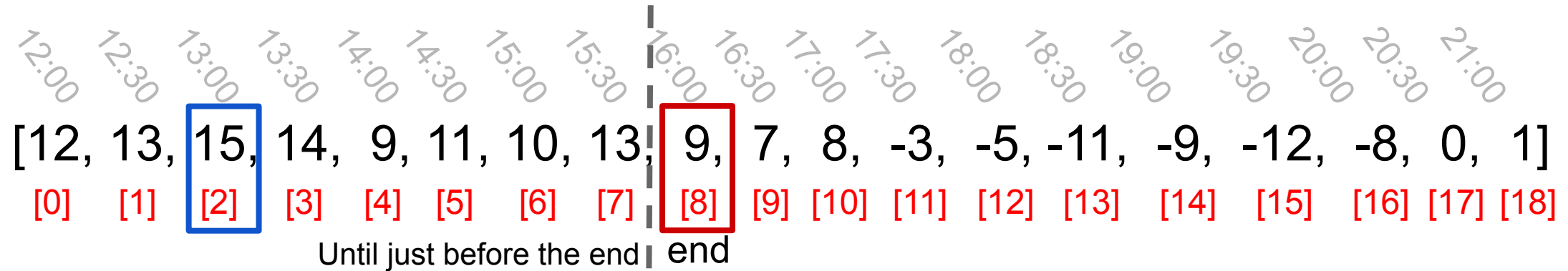
15

Indexing (1D Arrays) | Quick Quiz Answer



Let's use indexing to extract values from wind speed data every 30 minutes.

Array a: Wind speed data for every 30 minutes.



Q2. How would you extract the data from 13:00 to 16:00 (excluding 16:00)?

input

```
a[2: 8]
```

output

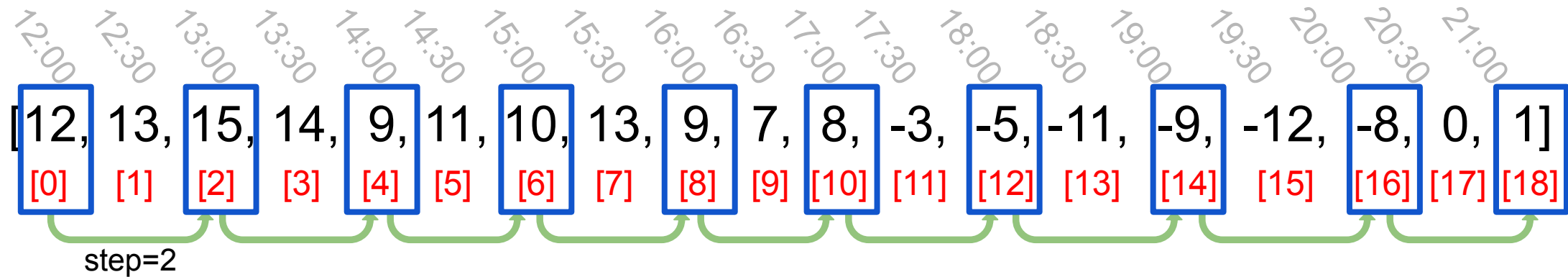
```
array([15, 14, 9, 11, 10, 13, 9])
```

Indexing (1D Arrays) | Quick Quiz Answer



Let's use indexing to extract values from wind speed data every 30 minutes.

Array a: Wind speed data for every 30 minutes.



Q3. How would you extract data for every 60 minutes?

input `a[:: 2]`

output `array([12, 15, 9, 10, 9, 8, -5, -9, -8, 1])`

2-Dimensional Arrays

N-dimensional Arrays

Represent N-dimensional array data with `numpy.ndarray`

- You can create an ndarray with more than two dimensions (N-dimensional).
- Similar to the 1-dimensional case, it is usually created as follows.

input

```
a = np.array([[5, 6, 8], [9, 2, 1]])
```


Nested* structure list

*Nesting structure: A structure where a list is inside another list

```
a = np.array([[5, 6, 8],  
              [9, 2, 1]])
```

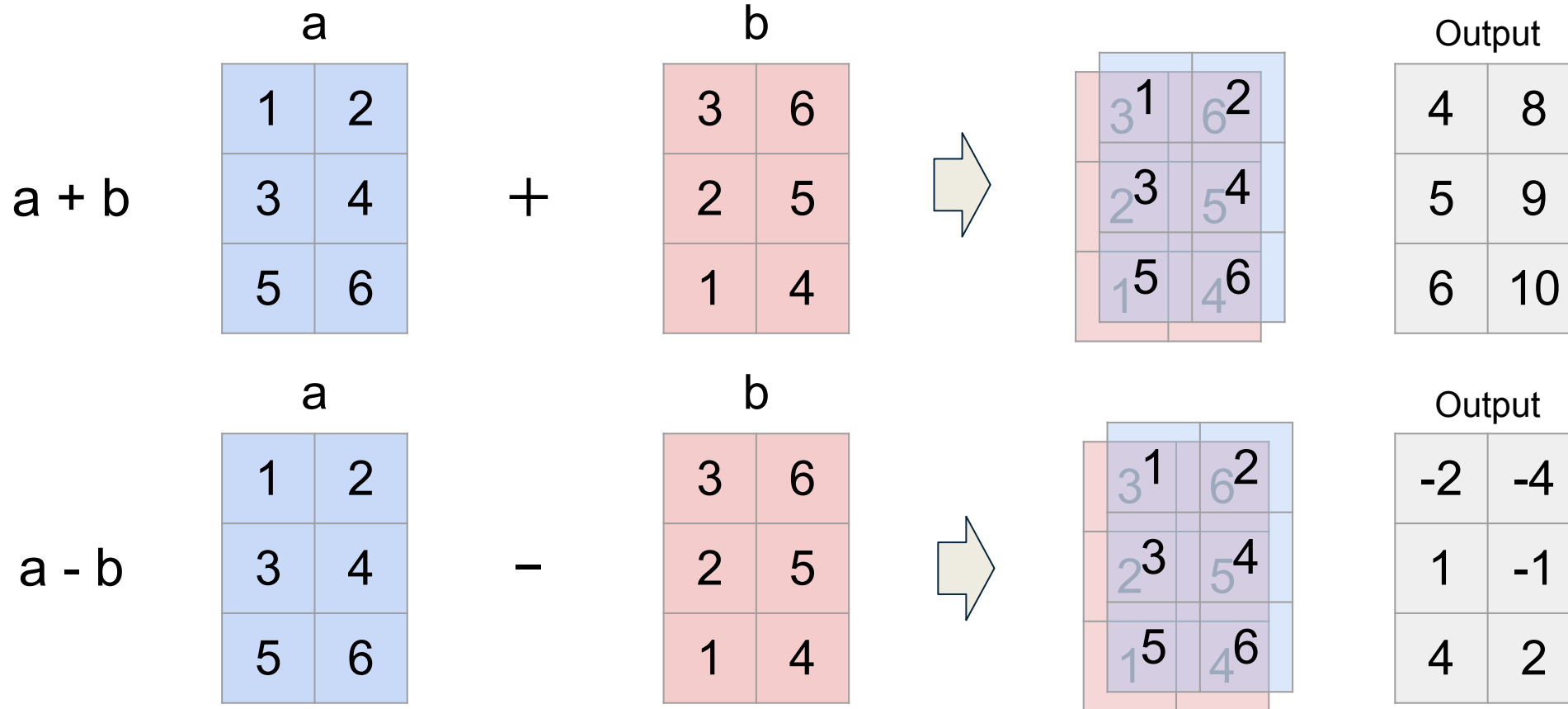
It is also okay to write code with line breaks as it does not affect execution

N-dimensional Arrays | Universal Function (ufunc)



Universal Function: A function that operates on each element of an ndarray

- Even with a 2-dimensional array, element-wise arithmetic operations can be performed

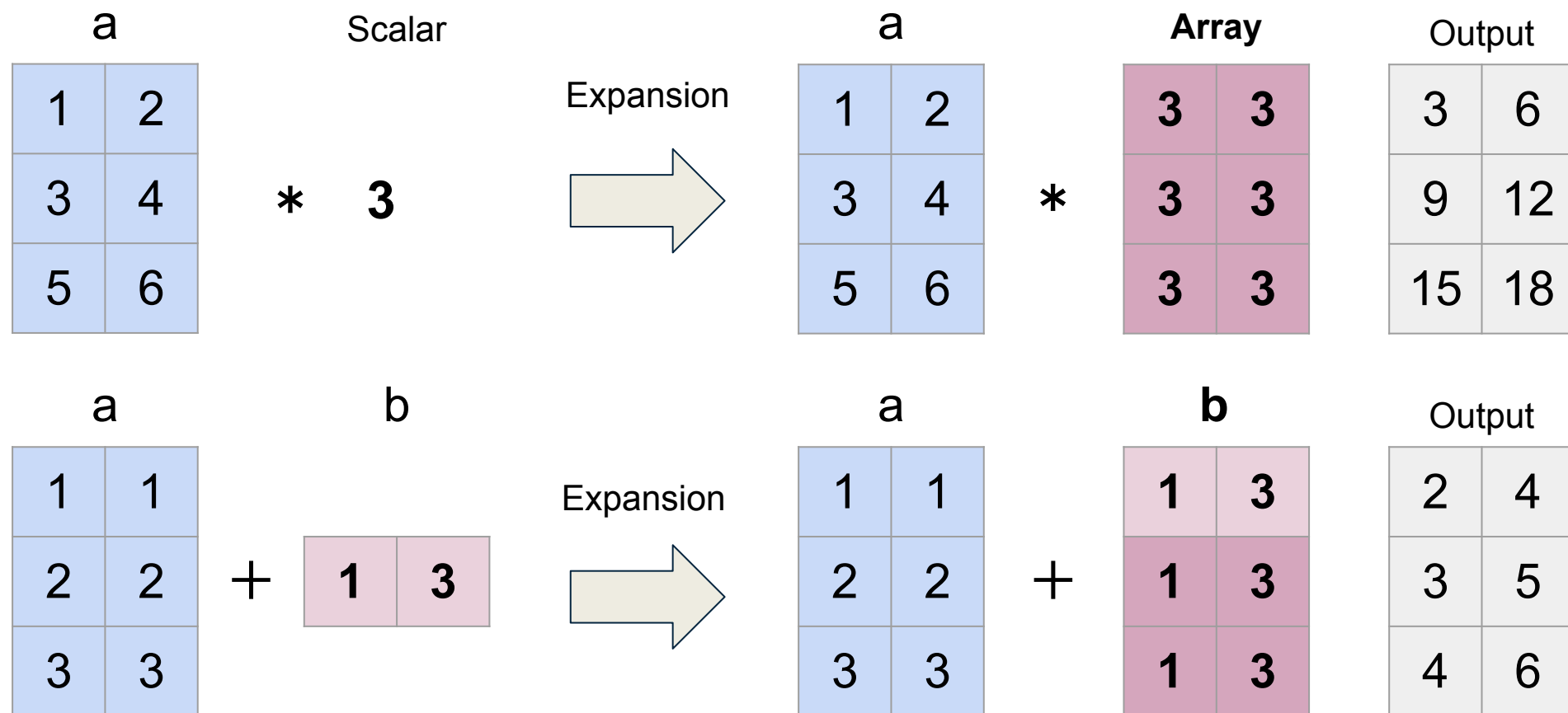


N-dimensional Arrays | Broadcasting



Broadcasting: A feature that automatically adjusts shapes to enable operations

- Dimensions with size 1 are extended to match the corresponding dimension sizes of the other array.

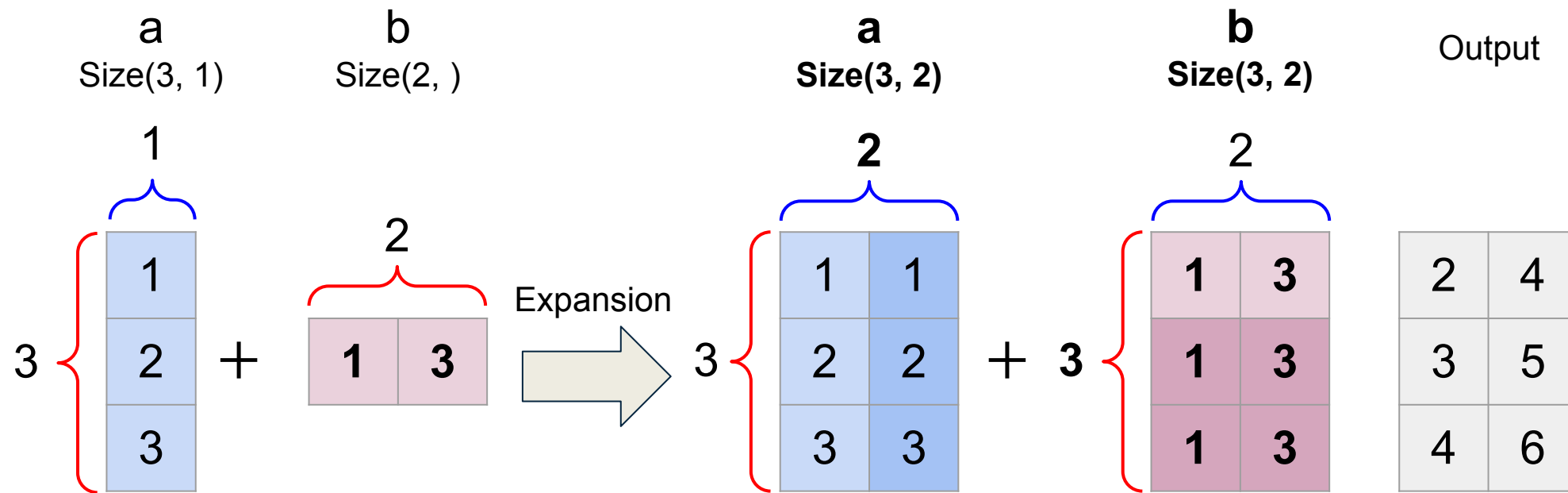


N-dimensional Arrays | Broadcasting



Broadcasting: A feature that automatically adjusts shapes to enable operations

- Dimensions with size 1 are extended to match the corresponding dimension sizes of the other array.



*For a 1-dimensional array, a dimension is added internally, and it is first converted from (2,) to a 2-dimensional array (1, 2) before expansion

Axis of an N-dimensional Array

Axis: A concept to easily access a specific direction in an N-dimensional array

- In ndarray, the dimension is distinguished as **axis**.
- In the case of a 2-dimensional array, **axis=0 represents rows**, and **axis=1 represents columns**.

1-dimensional array

axis = 0

[0] [1] [2]

[70, 95, 80]

2-dimensional array

[0]

[1]

[2]

axis = 1

[[70, 95, 80],

[34, 42, 50],

[74, 37, 65]]

[0]

[1]

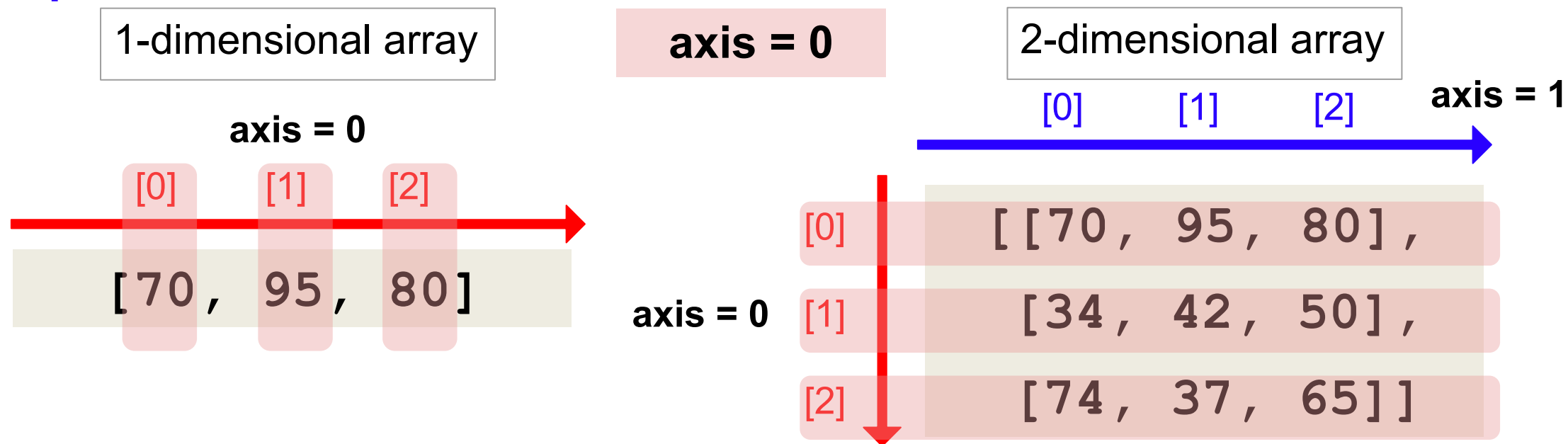
[2]

axis = 0

Axis of an N-dimensional Array

Axis: A concept to easily access a specific direction in an N-dimensional array

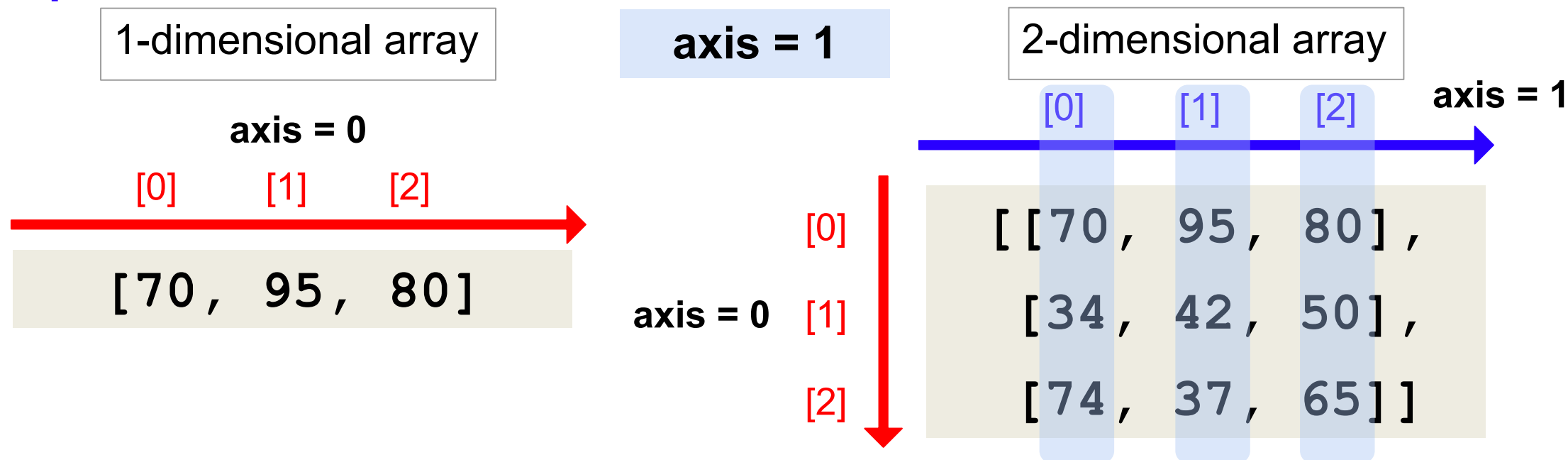
- In ndarray, the dimension is distinguished as **axis**.
- In the case of a 2-dimensional array, **axis=0 represents rows**, and **axis=1 represents columns**.



Axis of an N-dimensional Array

Axis: A concept to easily access a specific direction in an N-dimensional array

- In ndarray, the dimension is distinguished as **axis**.
- In the case of a 2-dimensional array, **axis=0 represents rows**, and **axis=1 represents columns**.

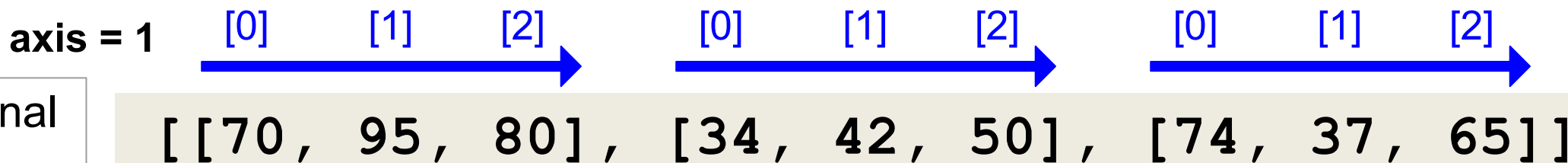
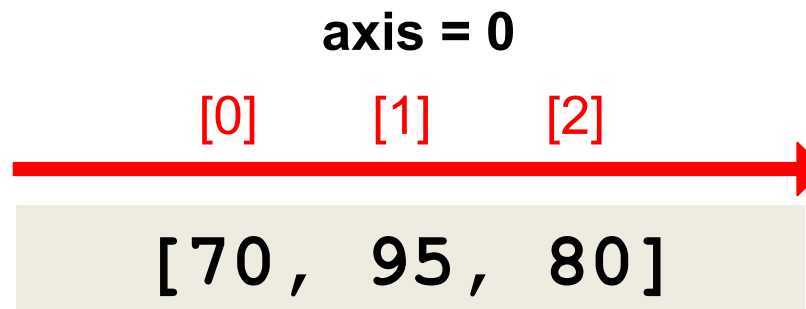


Axis of an N-dimensional Array



It is easier to understand when written flat!

1-dimensional
array



2-dimensional
array

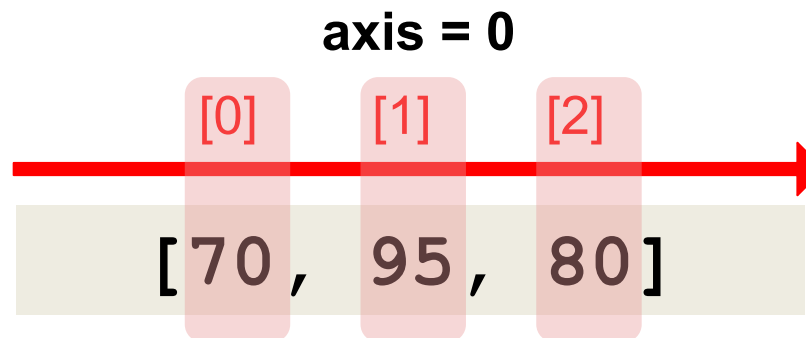
Axis of an N-dimensional Array



It is easier to understand when written flat!

axis = 0

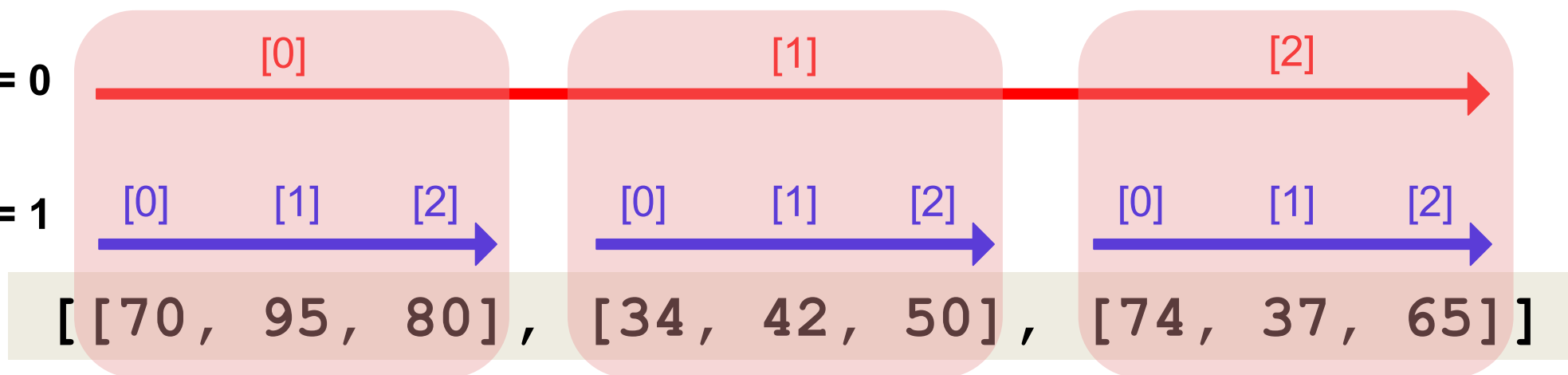
1-dimensional
array



axis = 0

axis = 1

2-dimensional
array



Axis of an N-dimensional Array



It is easier to understand when written flat!

axis = 1

1-dimensional
array

axis = 0

[0] [1] [2]

[70, 95, 80]

axis = 0

[0]

[1]

[2]

axis = 1

[0]

[1]

[2]

[0]

[1]

[2]

[0]

[1]

[2]

2-dimensional
array

[[70, 95, 80], [34, 42, 50], [74, 37, 65]]

Aggregation Function



A function that aggregates the elements within a NumPy array into a single value

Let's use the aggregation function to find the maximum value for each dimension.

2-dimensional array

	A [0]	B [1]	C [2]	D [3]
English [0]	75	60	82	80
Math [1]	90	73	75	85
Spanish [2]	82	92	80	63

Q1. What are A and D's highest scores?

Q2. What are the highest scores for each subject?



Check with an aggregation function

Aggregation Function



Use `np.max(ndarray)` to get the maximum value in the entire array.

2-dimensional
array

axis = 1

[0] [1] [2] [3]

	[0]	[1]	[2]	[3]
[0]	75	60	82	80
[1]	90	73	75	85
[2]	82	92	80	63

input

`np.max(a)`

output

92

axis = 0

Aggregation Function



Use `np.max(ndarray, axis=0)` to get the maximum value along the axis 0 direction.

2-dimensional
array

axis = 1

[0] [1] [2] [3]

[0]

75 60 82 80

[1]

90 73 75 85

[2]

82 92 80 63

axis = 0

`np.max(a,
axis=0)`

90 92 82 85

aggregate along
axis=0 direction

input

`np.max(a)`

output

92

input

`np.max(a, axis=0)`

output

`array([90, 92, 82, 85])`

Aggregation Function



Use `np.max(ndarray, axis=1)` to get the maximum value along the axis 1 direction.

2-dimensional
array

axis = 1

[0] [1] [2] [3]

`np.max(a,
axis=1)`

input

`np.max(a)`

output

92

input

`np.max(a, axis=0)`

output

`array([90, 92, 82, 85])`

input

`np.max(a, axis=1)`

output

`array([82, 90, 92])`

[0]

75

60

82

80

82

[1]

90

73

75

85

90

[2]

82

92

80

63

92

aggregate along
axis=1 direction

Aggregation Function



Other aggregation functions: `np.min()`, `np.sum()`, `np.mean()`, `np.std()`, etc.

2-dimensional
array

axis = 1

[0] [1] [2] [3]

Minimum value

input

`np.min(a, axis=0)`

output

`array([75, 60, 75, 63])`

Sum

input

`np.sum(a, axis=0)`

Mean

input

`np.mean(a, axis=0)`

Standard deviation

input

`np.std(a, axis=0)`

[0]

[1]

[2]

75	60	82	80
90	73	75	85
82	92	80	63

axis = 0

Indexing (2D Arrays) | Quick Quiz



Let's extract specific elements from the 2D array a using various indexing methods.

axis = 1

2-dimensional
array

A B C D
[0] [1] [2] [3]

English [0]

Math [1]

Spanish [2]

axis = 0

75	60	82	80
90	73	75	85
82	92	80	63

Q1. What is D's math grades?

Q2. What are A, B, and C's grades in English and Spanish?



**Extract using
indexing**

Recap: Axis of an N-dimensional Array



It is easier to understand when written flat!

1-dimensional
array

axis = 0

[0] [1] [2]

[70, 95, 80]

axis = 0

[0]

[1]

[2]

axis = 1

[0]

[1]

[2]

[0]

[1]

[2]

[0]

[1]

[2]

2-dimensional
array

[[70, 95, 80], [34, 42, 50], [74, 37, 65]]

Indexing (N-D Arrays) | Basic Index Specification



Extract elements by specifying indexes from lower dimensions (axis 0=0, axis 1=1)

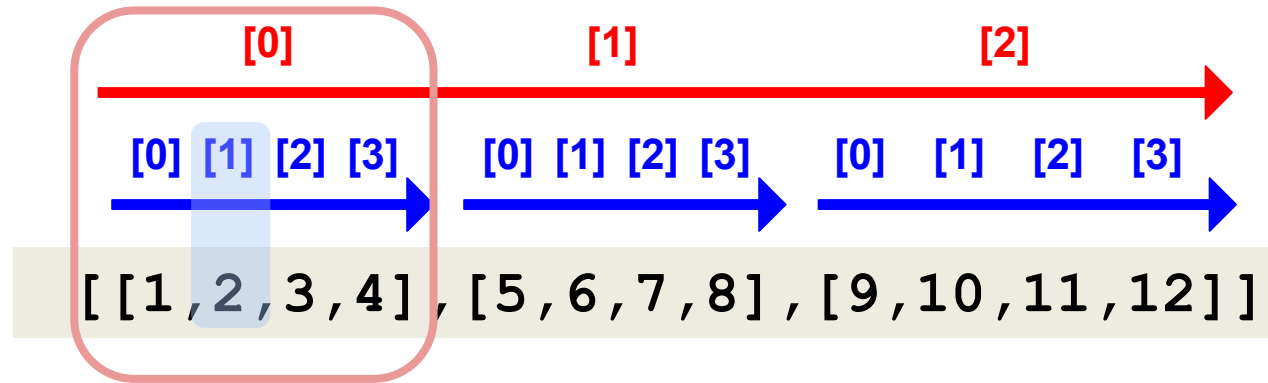
2-dimensional
array a

		axis=1			
		[0]	[1]	[2]	[3]
axis=0	[0]	1	2	3	4
	[1]	5	6	7	8
	[2]	9	10	11	12

Desired output

[0]

2



axis 0=0 axis 1=1

input

a[0, 1]

output

2

Indexing (N-D Arrays) | Basic Index Specification



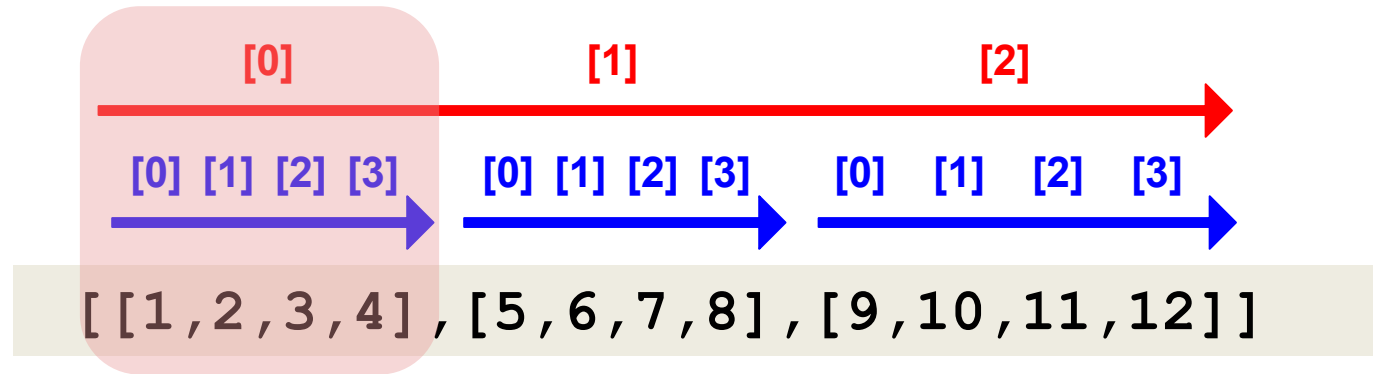
Extract elements from a 2D array along the axis=0 direction using slicing.

2-dimensional
array a

	axis=1			
	[0]	[1]	[2]	[3]
axis=0 [0]	1	2	3	4
[1]	5	6	7	8
[2]	9	10	11	12

Desired output

[0]	[1]	[2]	[3]
1	2	3	4



↓ Use slicing (:), as the indexes are consecutive

input

`a[0]` (same as `a[0, :]`)

output

`array([1, 2, 3, 4])`

Indexing (N-D Arrays) | Basic Index Specification



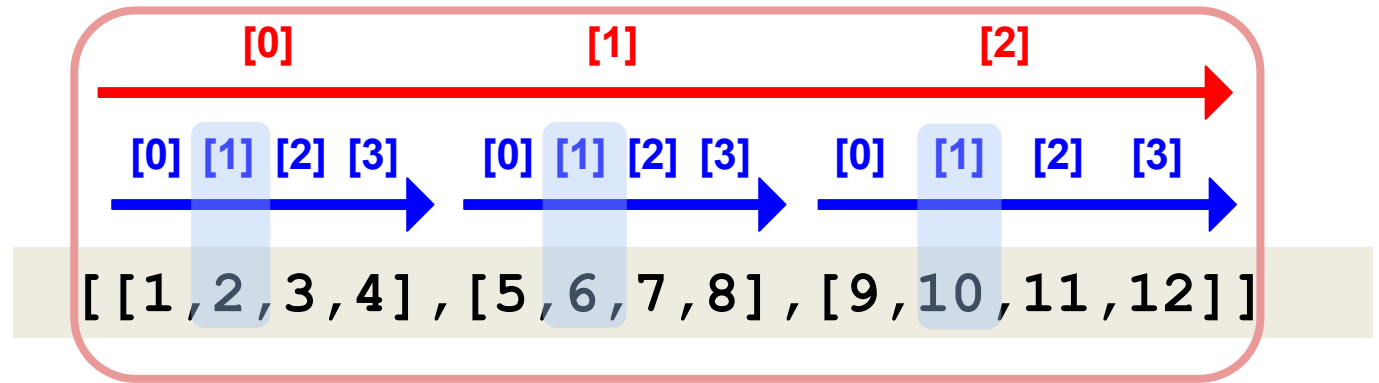
Extract elements from a 2D array along the axis=1 direction using slicing.

2-dimensional
array a

		axis=1			
		[0]	[1]	[2]	[3]
axis=0	[0]	1	2	3	4
	[1]	5	6	7	8
	[2]	9	10	11	12

Desired output

[0]	[1]	[2]
2	6	10



↓ Use slicing (:), as the indexes are consecutive

input

```
a[:, 1]
```

output

```
array([2, 6, 10])
```

Indexing (N-D Arrays) | Basic Index Specification



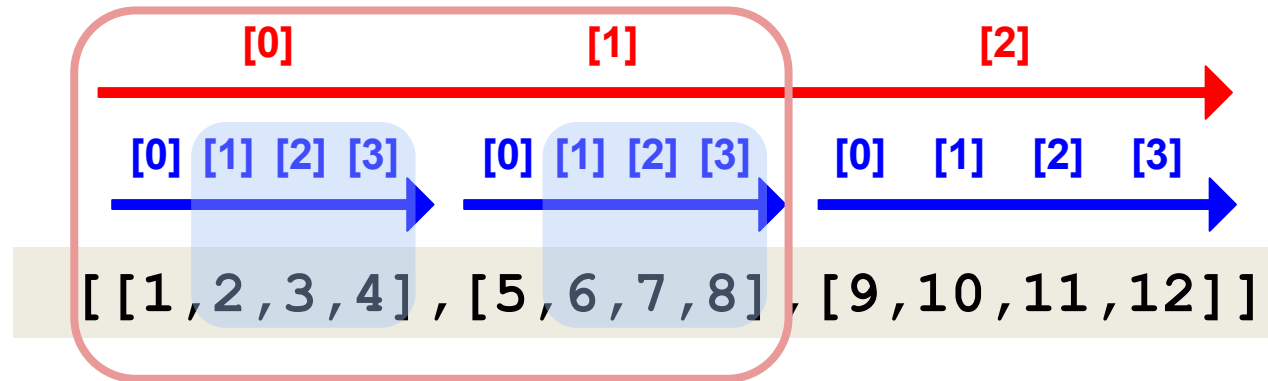
Combine slicing to extract consecutive elements along multiple axes.

2-dimensional
array a

		axis=1			
		[0]	[1]	[2]	[3]
axis=0	[0]	1	2	3	4
	[1]	5	6	7	8
	[2]	9	10	11	12

Desired output

		[0]	[1]	[2]
axis=0	[0]	2	3	4
	[1]	6	7	8



input

`a[:2, 1:]`

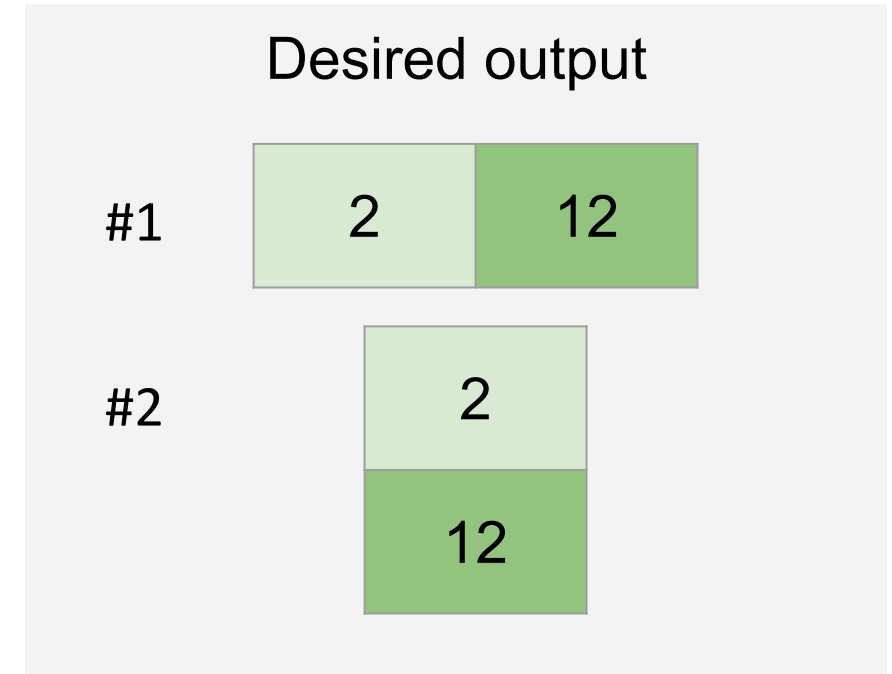
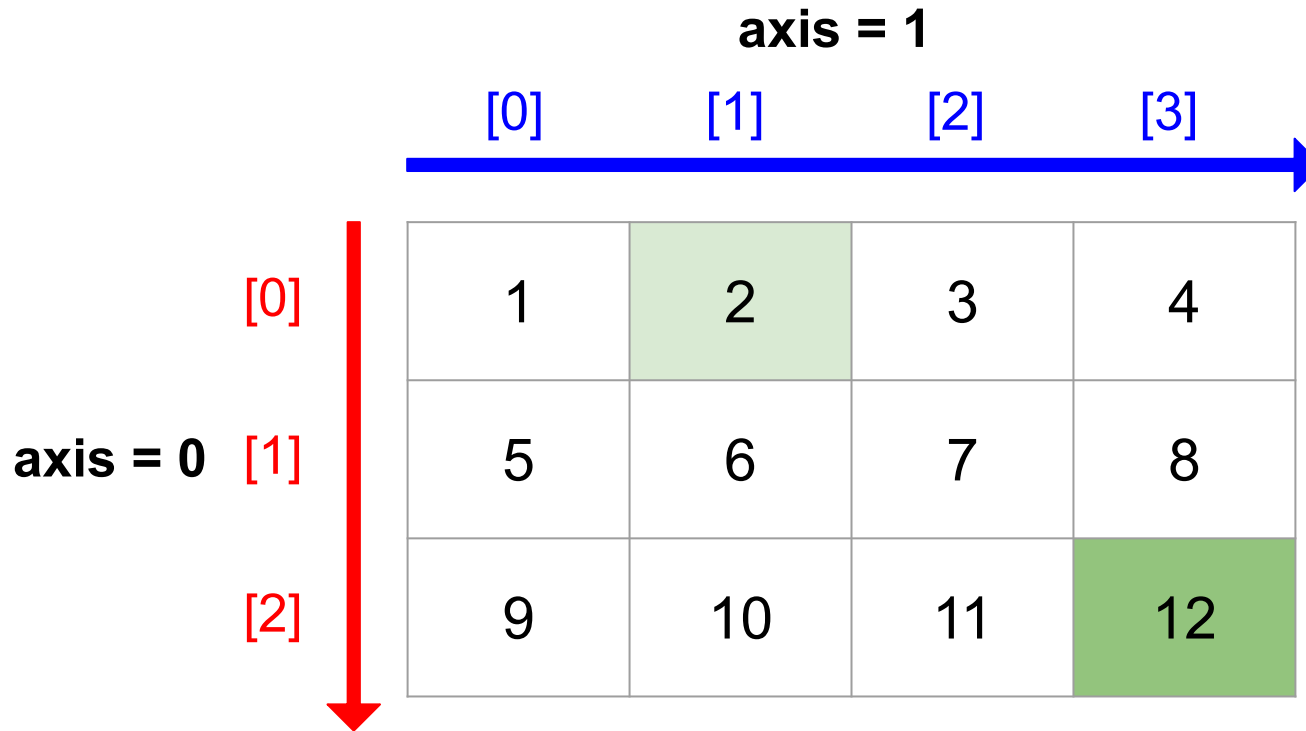
output

`array([[2, 3, 4], [6, 7, 8]])`

Indexing (N-D Arrays) | Advanced Index Specification



To extract elements at **distant positions** in **any order**, how should it be done?



Indexing (2) | Rules of Advanced Index Specification



Apply the following rules to retrieve distant elements in any order:

1. Specify for each axis like `a[[index of axis=0], [index of axis=1]]`
2. “Shape” and “positions of elements” of the output ndarray match index specification.
3. Omitting notation by using broadcasting.

Indexing (N-D Arrays) | Advanced Index Specification



1: Specify for each axis like a[[index of axis=0], [index of axis=1]]

2-dimensional
array a

		axis=1			
		[0]	[1]	[2]	[3]
axis=0	[0]	1	2	3	4
	[1]	5	6	7	8
	[2]	9	10	11	12

Desired output

	[0]	[1]
	2	12

index of axis=0

index of axis=1

a[0 2 , 1 3]

input

a[[0, 2], [1, 3]]

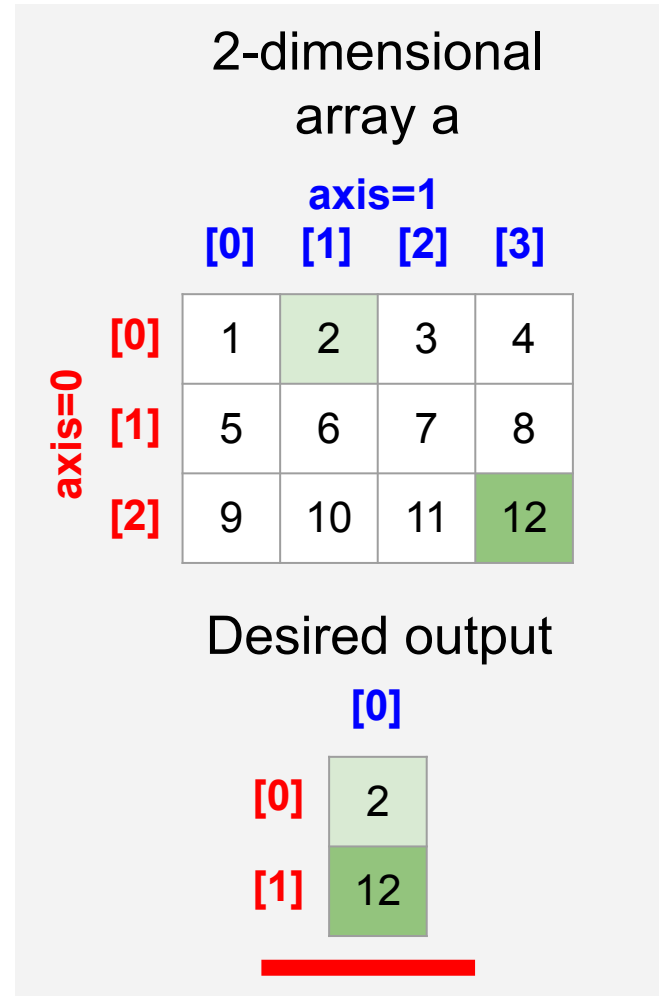
output

array([2, 12])

Indexing (N-D Arrays) | Advanced Index Specification



2: “Shape” and “positions of elements” of the output ndarray match index specification.



index of axis=0 index of axis=1

a[

0
2

 ,

1
3

]

input a[

[0]

 ,

[2]

 ,

[1]

 ,

[3]

]

output array([

[2]

 ,

[12]

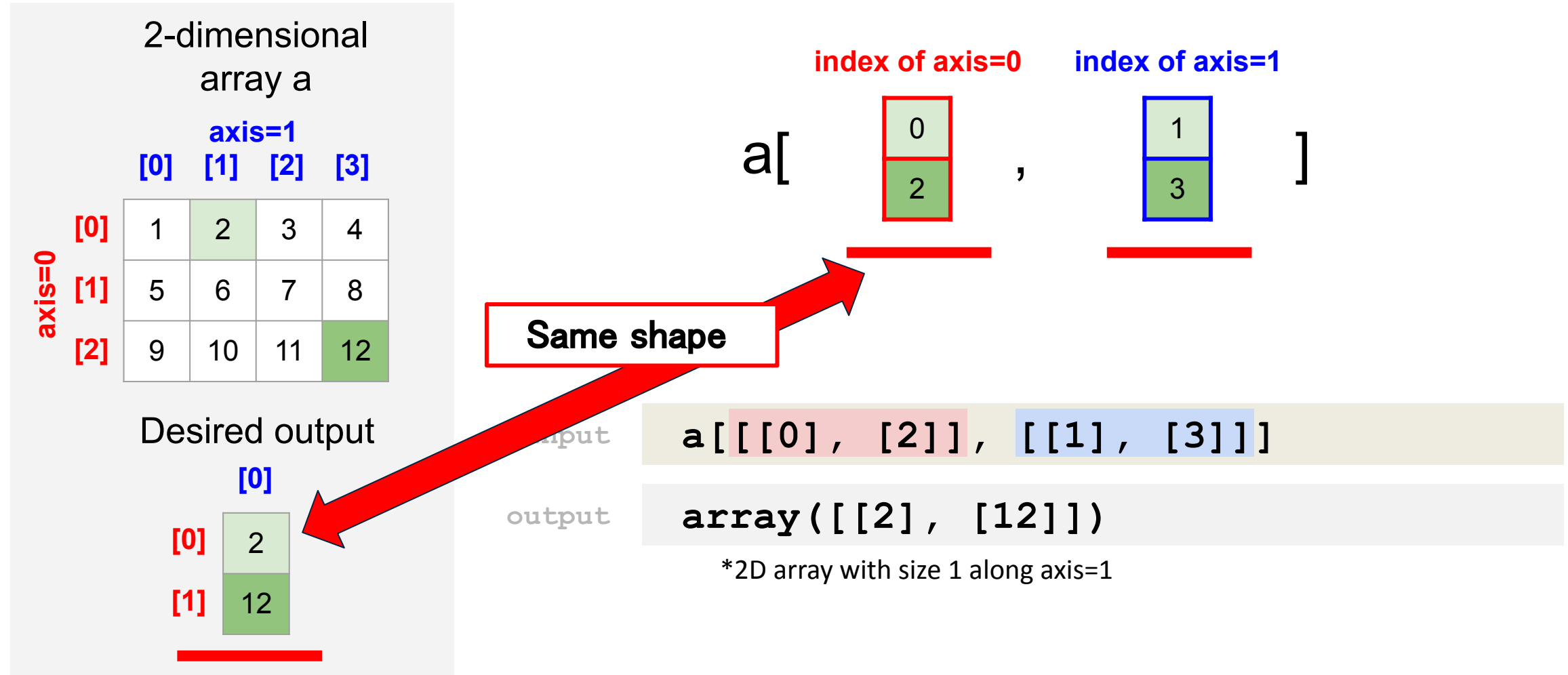
])

*2D array with size 1 along axis=1

Indexing (N-D Arrays) | Advanced Index Specification



2: “Shape” and “positions of elements” of the output ndarray match index specification.



Indexing (N-D Arrays) | Advanced Index Specification



3: Omitting notation by using broadcasting.

2-dimensional
array a

		axis=1			
		[0]	[1]	[2]	[3]
axis=0	[0]	1	2	3	4
	[1]	5	6	7	8
	[2]	9	10	11	12

Desired output

[0]	[1]
2	4

*Broadcasting: A feature that automatically adjusts shapes to enable operations

index of axis=0 index of axis=1

a[0 0 , 1 3] Omission a[0 , 1 3]

input

a[[0, 0] , [1, 3]]

a[0 , [1, 3]]

output

array([2, 4])

array([2, 4])

Indexing (N-D Arrays) | Advanced Index Specification



3: Omitting notation by using broadcasting.

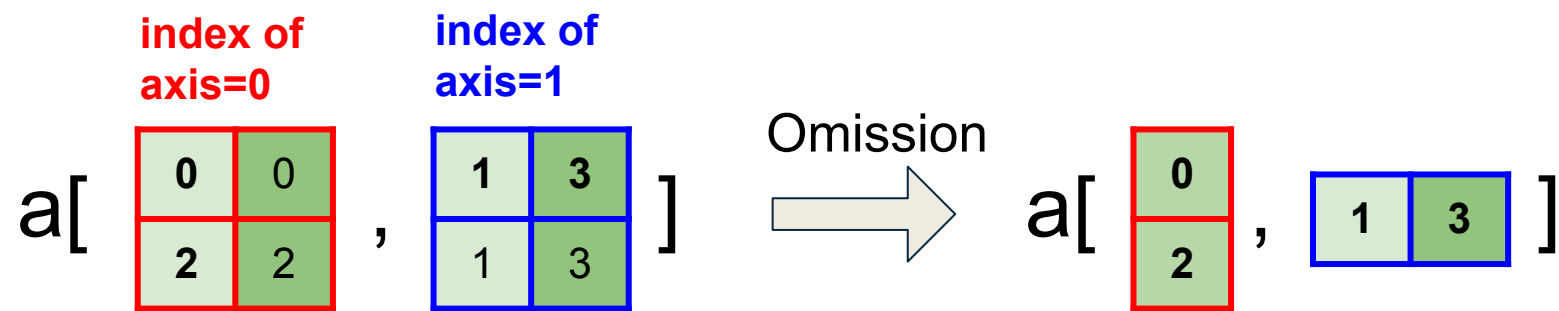
2-dimensional
array a

	axis=1			
	[0]	[1]	[2]	[3]
[0]	1	2	3	4
[1]	5	6	7	8
[2]	9	10	11	12

Desired output

	[0]	[1]
[0]	2	4
[1]	10	12

*Broadcasting: A feature that automatically adjusts shapes to enable operations



input `a[[[0, 0], [2, 2]], [[1, 3], [1, 3]]]`

output `array([[2, 4], [10, 12]])`

input `a[[[0], [2]], [1, 3]]`

output `array([[2, 4], [10, 12]])`

Indexing (N-D Arrays) | Basic + Advanced Index Specification



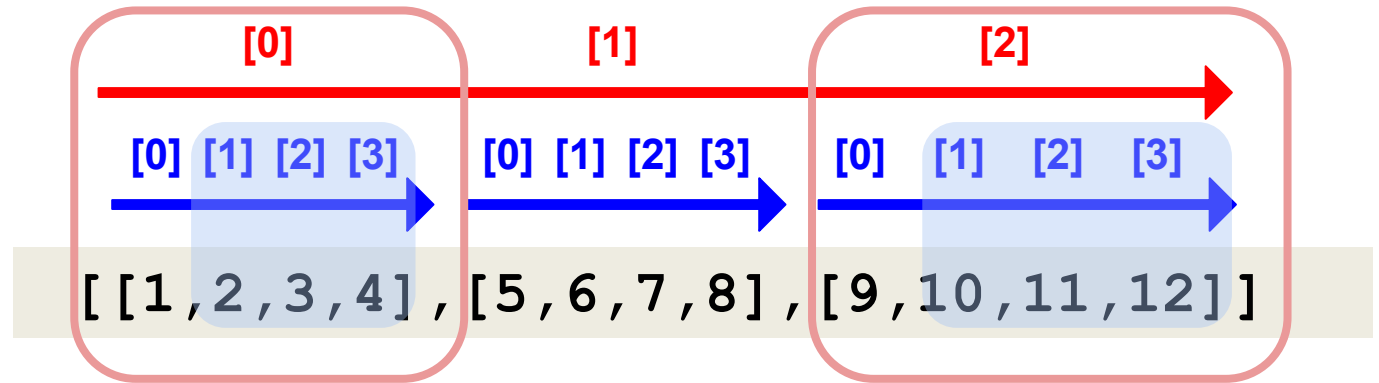
You can also combine basic index specification with advanced index specification.

2-dimensional
array a

		axis=1			
		[0]	[1]	[2]	[3]
axis=0	[0]	1	2	3	4
	[1]	5	6	7	8
	[2]	9	10	11	12

Desired output

		[0]	[1]	[2]
axis=0	[0]	2	3	4
	[1]	10	11	12



	↓Advanced specification	↓Basic specification
input	<code>a[[0, 2],</code>	<code>1:]</code>
output	<code>array([[2, 3, 4], [10, 11, 12]])</code>	

Indexing (2D Arrays) | Quick Quiz Answer



Let's extract specific elements from the 2D array a using various indexing methods.

2-dimensional array

axis = 1

A B C D

[0] [1] [2] [3]

English [0]

Math [1]

Spanish [2]

axis = 0

75	60	82	80
90	73	75	85
82	92	80	63

Q1. What is D's math grades?

input a[1, 3]

output 85

Indexing (2D Arrays) | Quick Quiz Answer



Let's extract specific elements from the 2D array `a` using various indexing methods.

axis = 1

2-dimensional
array

A B C D
[0] [1] [2] [3]

English [0]

Math [1]

Spanish [2]

axis = 0

75	60	82	80
90	73	75	85
82	92	80	63

Q2. What are A, B, and C's grades in English and Spanish?

input

```
a[[0, 2], :3]
```

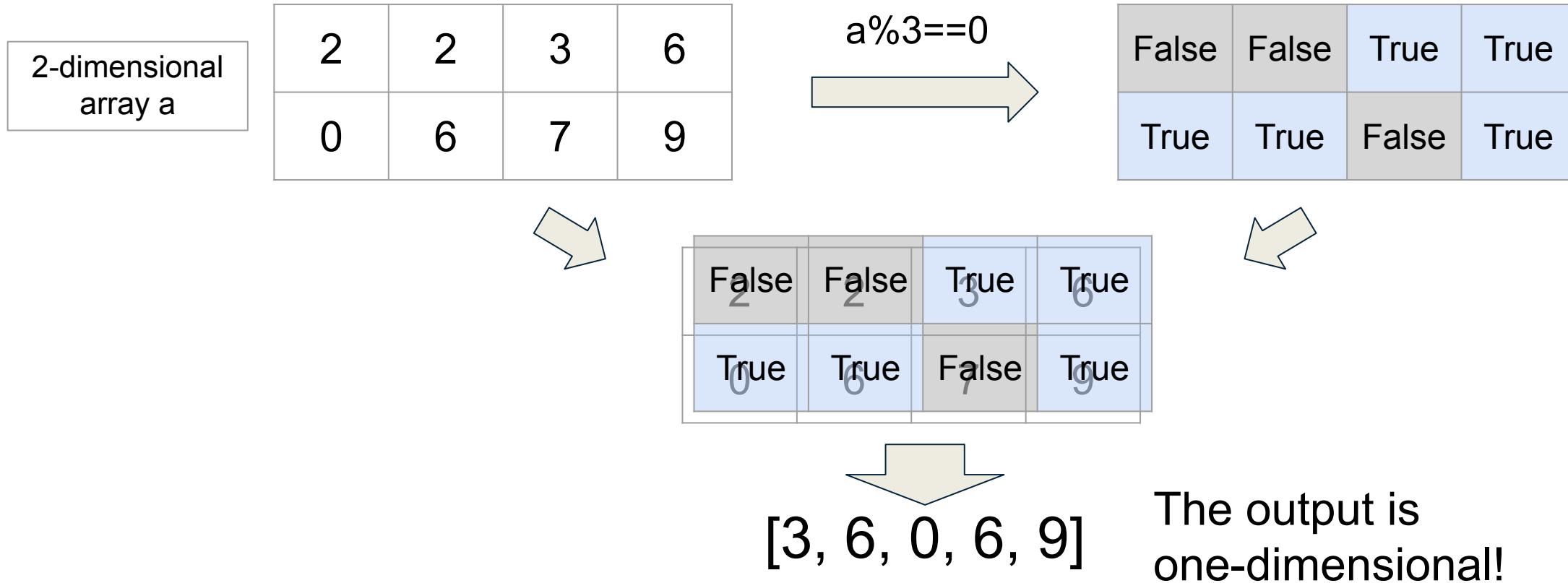
output

```
array([[75, 60, 82],  
       [82, 92, 80]])
```

Indexing (N-D Arrays) | Boolean Indexing



- Boolean values can also be used to specify the values to be retrieved.



Numpy : A library for performing complex scientific computations in Python.

1. Creating an ndarray using `np.array()`
2. **Universal functions** allows performing element-wise calculations without using loops
3. Broadcasting allows performing operations on arrays of different sizes
4. Understanding the concepts of indexing and **axis**(dimensions) in ndarrays
5. Extracting specific elements or subarrays from an array with indexing
6. Calculating statistics along each axis with aggregation functions



Appendix: np.zeros(), np.ones()



0.	0.	0.
0.	0.	0.

input

```
np.zeros ( (2, 3) )
```

output

```
array([[0., 0., 0.],  
       [0., 0., 0.]])
```

1.	1.	1.
1.	1.	1.

input

```
np.ones ( (2, 3) )
```

output

```
array([[1., 1., 1.],  
       [1., 1., 1.]])
```

*The dot (.) after 0 or 1 indicates that the data type is float (floating-point number).

Appendix: np.arange()

`np.arange(start, stop, step)`: Generates an ndarray from start to stop with a step interval

- Only the stop can be specified, while start and step can be omitted.

input

```
np.arange(8)
```

output

```
array([0, 1, 2, 3, 4, 5, 6, 7])
```

input

```
np.arange(2, 8)
```

output

```
array([2, 3, 4, 5, 6, 7])
```

input

```
np.arange(2, 8, 2)
```

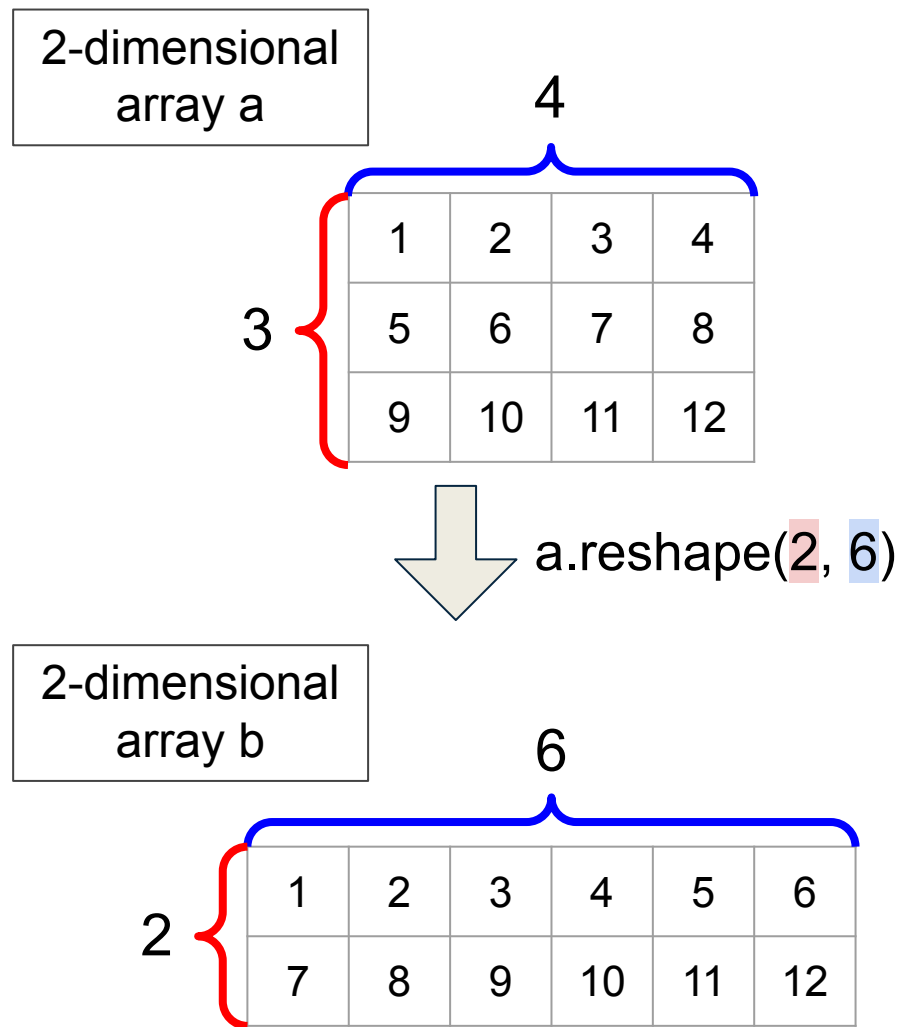
output

```
array([2, 4, 6])
```

Appendix: np.reshape()



`np.reshape(array, new_shape)`: A function to change the shape of an ndarray.



input `a.shape`

output `(3, 4)`

It can be specified like:

`ndarray.reshape( size of axis=0 , size of axis=1 )`

input `b = a.reshape(2, 6)`
`b.shape`

output `(2, 6)`

Appendix: np.reshape()



When the number of elements and the size of other dimensions are known, **specifying -1 automatically adjusts the size.**

2-dimensional
array a

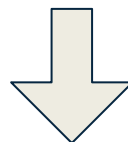
4			
1	2	3	4
5	6	7	8
9	10	11	12

input

`a.shape`

output

`(3, 4)`



`a.reshape(2, -1)`

Specifying (2, -1) automatically converts it to (2, 6)

2-dimensional
array b

6					
1	2	3	4	5	6
7	8	9	10	11	12

input

`b = a.reshape(2, -1)`
`b.shape`

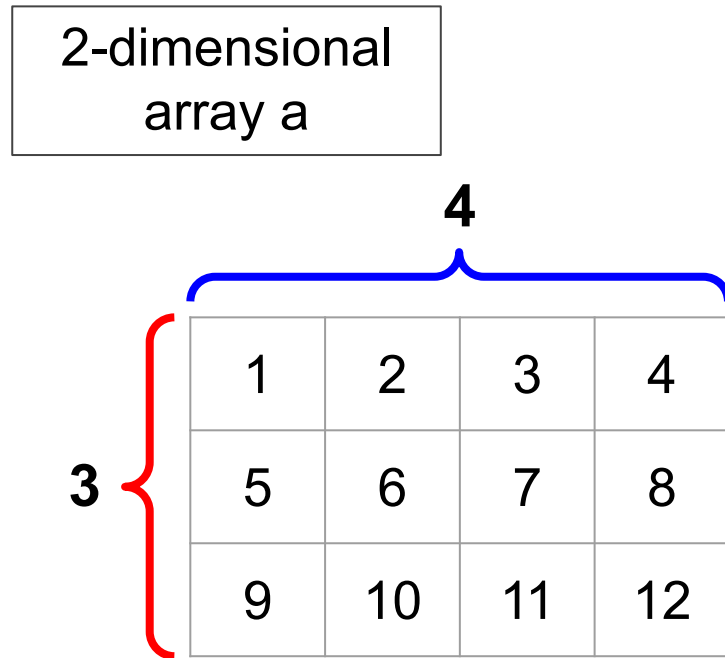
output

`(2, 6)`

Appendix: Attribute



Attribute : Information associated with an ndarray



ndarray.shape: Indicates the shape of the array

input

a.shape

output

(3, 4)

ndarray.ndim: Indicates the dimension of the array

input

a.ndim

output

2

ndarray.dtype: indicates the data type of the array

input

a.dtype

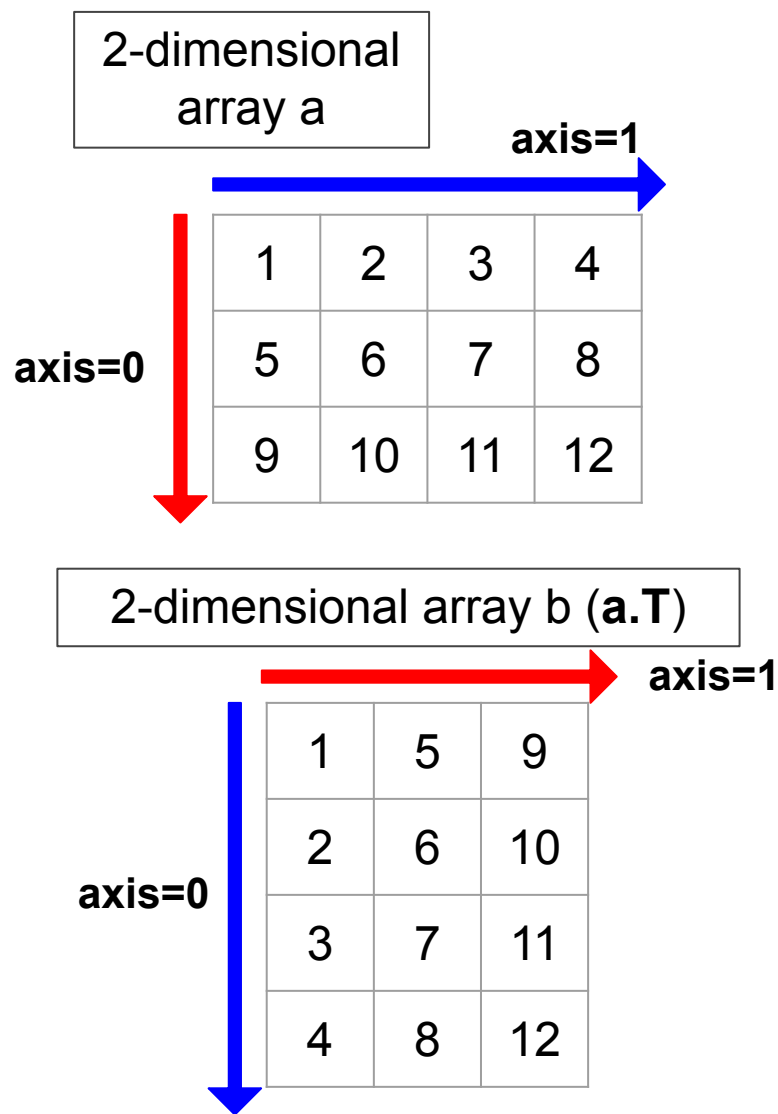
output

dtype('int64')

*int64: 64-bit integer type

Appendix: ndarray.T (Transpose)

`ndarray.T` : Tranpose. An operation that swaps the rows and columns of an array.



input

```
b = a.T  
b
```

output

```
array([[ 1,  5,  9],  
       [ 2,  6, 10],  
       [ 3,  7, 11],  
       [ 4,  8, 12]])
```

input

```
b.shape
```

output

```
(4, 3)
```